

Master's Thesis

Practical Regularized Quasi-Newton
Methods with Inexact Function Values
(不正確な関数値下における
実用的な正則化準ニュートン法)

48-246226 Hiroki Hamaguchi

Supervisor Professor Takeda Akiko

February, 2026

Department of Mathematical Informatics
Graduate School of Information Science and Technology
The University of Tokyo

Abstract

Many practical optimization problems involve objective function values that are corrupted by unavoidable numerical errors. In smooth unconstrained optimization, quasi-Newton methods combined with line search are widely used due to their efficiency and scalability. These methods implicitly assume accurate function evaluations and thus they may fail to converge in noisy settings. Developing fast and robust quasi-Newton methods for such scenarios is crucial.

To address this issue, we propose a noise-tolerant regularized quasi-Newton method with an Armijo-type line search. The regularization parameter is updated using an Objective-Function-Free Optimization inspired rule that is closely related to the AdaGrad-Norm method. Under an error model of the objective function and standard assumptions, we establish a global convergence rate of $\mathcal{O}(1/\varepsilon^2)$ for reaching a first-order stationary point, matching the standard convergence rate for first-order methods. We performed extensive experiments on the CUTEst benchmark collection with artificially noisy objective function evaluations, as well as with low-precision floating-point arithmetic (64-, 32-, and 16-bit). The results show that the proposed method is substantially more robust than several existing methods, while maintaining competitive practical convergence speed and computational cost.

Keywords Regularized Quasi-Newton Method, Numerical Stability, Numerical Error, AdaGrad-Norm, Objective-Function-Free Optimization

Contents

Chapter 1	Introduction	1
1.1	Introduction	1
1.2	Organization of This Thesis	2
Chapter 2	Foundations of Quasi-Newton Methods	3
2.1	Quasi-Newton Methods	3
2.2	Modified Secant Condition	4
Chapter 3	Noise-Tolerant Regularized Quasi-Newton Method	10
3.1	Problem Setting and Preliminaries	10
3.2	Proposed Algorithm	12
3.3	Theoretical Analysis	13
3.4	Practical Choices of Algorithmic Variables	21
3.5	Numerical Experiments	24
3.6	Conclusion	29
	Acknowledgments	30
A	Appendix	35
A.1	Proof of Lemma 5	35
A.2	Proof of Proposition 2	35

Chapter 1

Introduction

1.1 Introduction

We consider the unconstrained optimization problem

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \quad f(x), \tag{1.1}$$

where $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is a continuously differentiable nonconvex function, and $x \in \mathbb{R}^n$ is the optimization variable. We focus on settings where objective function evaluations are contaminated by bounded and non-diminishing numerical noise [46]. Specifically, only inexact function values $\bar{f}(x)$ are available for optimization. Such inaccuracies are unavoidable in many realistic scenarios, including finite-precision arithmetic, simulation-based evaluations, and stochastic approximations [9, 36, 45]. When function values are contaminated by numerical noise, standard optimization algorithms may lose their theoretical guarantees, behave erratically, or terminate prematurely with unreliable solutions.

In the absence of noise and with accurate function evaluations, quasi-Newton methods, in particular the Broyden–Fletcher–Goldfarb–Shanno (BFGS) method and its Limited-memory variant (L-BFGS), are among the most efficient and widely used algorithms to solve the problem (1.1) [31, 38]. Their success is largely attributed to fast local convergence and scalability. Standard implementations [39, 49] rely on line search procedures that enforce the Wolfe conditions to guarantee sufficient descent and the positive definiteness of the Hessian approximation.

In the presence of numerical noise, these line search conditions may become unreliable. Differences in function values or directional derivatives can be dominated by rounding errors, leading to unstable step sizes, ill-conditioned Hessian approximations, or abnormal termination [40, 45]. From a computational perspective, improving the robustness of quasi-Newton methods in noisy environments is therefore crucial. In particular, it is desirable to design algorithms that (i) retain computational efficiency when function evaluations are accurate, (ii) remain stable when function values are unreliable, and (iii) guarantee convergence to first-order stationary points under mild assumptions.

In this work, we propose a noise-tolerant regularized quasi-Newton method that addresses these challenges. Our approach combines Armijo-type line search with quadratic regularization [27, 48] and Objective-Function-Free Optimization (OFFO) inspired strategy [22, 23]. When a sufficient decrease in function value is observed, the method behaves similarly to standard line-search-based quasi-Newton methods, preserving its practical efficiency. When numerical noise dominates, we introduce regularization on the approximate Hessian to ensure numerical stability, which yields convergence properties similar to those of OFFO methods, particularly AdaGrad-Norm [51]. The resulting algorithm is particularly well-suited for low-precision or noisy environments.

From a theoretical standpoint, we establish that the proposed method achieves the standard global convergence rate of $\mathcal{O}(1/\varepsilon^2)$ for smooth nonconvex optimization problems, even when objective function values are corrupted by noise. Extensive numerical experiments on the CUTest

benchmark collection, across various floating-point precisions and artificial noise settings, demonstrate that the proposed method is significantly more stable than standard line-search-based methods in settings with significant numerical noise, while maintaining practical convergence speed.

1.2 Organization of This Thesis

The remainder of this thesis is organized as follows.

Chapter 2 reviews the foundations of quasi-Newton methods. In section 2.1, we briefly explain what quasi-Newton methods are. In section 2.2, we explain the modified secant condition, which is used in our numerical experiments. Since both topics are standard and well documented in the literature, we present them as a separate chapter.

Chapter 3 presents the proposed regularized quasi-Newton method along with its theoretical analysis. In section 3.1, we introduce the problem setting and review relevant background. In section 3.2, we present the proposed algorithm. In section 3.3, we establish a global convergence property of the proposed algorithm. In section 3.4, we discuss practical choices of algorithmic variables and parameters. In section 3.5, we report numerical results on the CUTEst benchmark problems. Finally, in section 3.6, we conclude the thesis with a summary and future research directions.

Our implementation is publicly available on GitHub^{*1}.

^{*1} <https://github.com/HirokiHamaguchi/qnlab>

Chapter 2

Foundations of Quasi-Newton Methods

In this chapter, we review the foundations of quasi-Newton methods.

2.1 Quasi-Newton Methods

In this section, we discuss quasi-Newton methods. Quasi-Newton methods are based on Newton's method, but aim to reduce its primary drawback, namely, the high computational cost of evaluating the Hessian matrix. Specifically, instead of using the true Hessian matrix $\nabla^2 f(x_k)$, an approximation matrix B_k is employed, thereby achieving fast convergence while keeping the computational cost low.

Figure 2.1 shows a comparison of Newton's method, quasi-Newton methods, and gradient descent. Gradient descent is a method that simply updates in the direction opposite to the gradient. Although its per-iteration computational cost is the lowest, its convergence is slow. Newton's method converges in the fewest iterations, but has the highest computational cost per step. Quasi-Newton methods lie between these two extremes, striking a balance between them. In particular, when comparing methods in terms of computation time rather than the number of iterations, quasi-Newton methods often exhibit the best performance.

Line-search-based quasi-Newton methods generate a sequence $\{x_k\}_{k=0}^{\infty}$ converging to the optimal solution x^* as follows:

$$x_{k+1} = x_k - \alpha_k B_k^{-1} \nabla f(x_k) = x_k - \alpha_k H_k \nabla f(x_k).$$

Here, $\alpha_k > 0$ is the step size determined by line search, B_k is an approximation of the Hessian matrix $\nabla^2 f(x_k)$ at the point x_k , and $H_k := B_k^{-1}$ denotes its inverse.

The core of quasi-Newton methods lies in how to update B_k (or its inverse H_k) at each iteration so that it approaches the Hessian matrix $\nabla^2 f(x_k)$. Figure 2.2 illustrates this concept. First, around the current point x_k , a quadratic approximation model of the objective function f is constructed using B_k . Next, this quadratic model is minimized to obtain the next point x_{k+1} . After obtaining x_{k+1} , the approximation matrix B_k is updated to B_{k+1} using the gradient information at x_k and x_{k+1} . This procedure is repeated until convergence, which constitutes the quasi-Newton method.

2.1.1 Secant Condition

Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be of class C^2 . Suppose that a symmetric matrix B_k is given as an approximation of the Hessian matrix $\nabla^2 f(x_k)$. We determine an approximation Hessian B_{k+1} at the next point x_{k+1} . Although there are infinitely many possible candidates for such a matrix, it is natural to impose symmetry on B_{k+1} as well, given that the true Hessian matrix is symmetric. For the current point x_k and the next point x_{k+1} , we define the step and gradient difference as

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

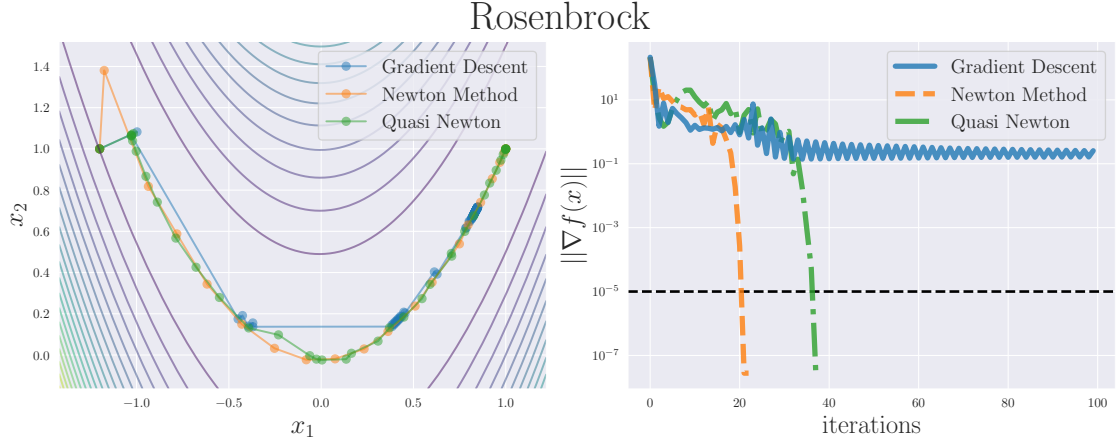
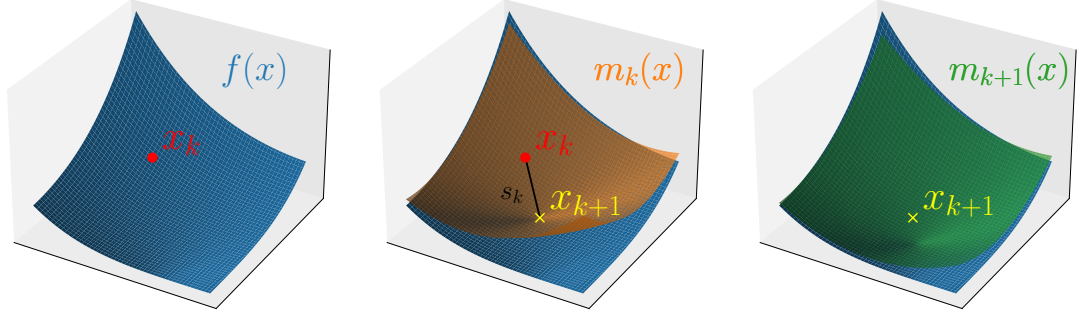


Fig. 2.1: Comparison of Newton's method, quasi-Newton methods, and gradient descent

Fig. 2.2: Conceptual illustration of quasi-Newton methods. (1) The objective function f (blue surface) and the current point x_k (red dot). (2) The quadratic model induced by the current Hessian approximation (orange surface) and its minimizer x_{k+1} (yellow cross). (3) The updated quadratic model based on the new point x_{k+1} (green surface).

We assume that $y_k^\top s_k \neq 0$ and $s_k \neq 0$. Here, the s in s_k stands for step. We can approximate $\nabla f(x_k)$ using the Hessian $\nabla^2 f(x_{k+1})$ and B_{k+1} as follows:

$$\begin{aligned}\nabla f(x_k) &\approx \nabla f(x_{k+1}) + \nabla^2 f(x_{k+1})(x_k - x_{k+1}) \\ &\approx \nabla f(x_{k+1}) + B_{k+1}(x_k - x_{k+1})\end{aligned}$$

Requiring the above approximation to hold as an equality yields

$$B_{k+1}(x_{k+1} - x_k) = \nabla f(x_{k+1}) - \nabla f(x_k),$$

or equivalently,

$$B_{k+1}s_k = y_k.$$

This relation is called the secant condition, or the quasi-Newton equation.

2.2 Modified Secant Condition

This section explains the modified secant condition, a modification of the standard secant condition used in quasi-Newton methods. The modified secant condition incorporates function value

information in addition to gradient information, leading to more accurate Hessian approximations.

2.2.1 Standard Secant Condition

First, recall that the standard secant condition used in quasi-Newton methods is given by

$$B_{k+1}s_k = y_k.$$

To justify this condition, consider a quadratic approximation model around x_{k+1} :

$$m_{k+1}(x) = f(x_{k+1}) + \nabla f(x_{k+1})^\top (x - x_{k+1}) + \frac{1}{2}(x - x_{k+1})^\top B_{k+1}(x - x_{k+1}).$$

By construction, this model satisfies

$$\begin{cases} m_{k+1}(x_{k+1}) = f(x_{k+1}) \\ \nabla m_{k+1}(x_{k+1}) = \nabla f(x_{k+1}) \end{cases}$$

regardless of the choice of B_{k+1} . Additionally, the secant condition ensures that the model satisfies

$$\begin{aligned} \nabla m_{k+1}(x_k) &= \nabla f(x_{k+1}) - B_{k+1}(x_{k+1} - x_k) && \text{(definition of model)} \\ &= \nabla f(x_{k+1}) - (\nabla f(x_{k+1}) - \nabla f(x_k)) && \text{(secant condition)} \\ &= \nabla f(x_k) \end{aligned}$$

which matches the gradient information from the previous iteration. Thus, by using the secant condition, we ensure that the quadratic model $m_{k+1}(x)$ accurately reflects the gradient information at both x_{k+1} and x_k , except for the previous function value $f(x_k)$.

2.2.2 Modified Secant Condition

Although the secant condition is widely used, it can fail to capture the curvature of the objective function accurately because it only matches gradient information. We illustrate this limitation in Figure 2.3.

In Figure 2.3a, we have two points x_k and x_{k+1} with their corresponding function values and gradients. In Figure 2.3b, we see that the ideal quadratic model constructed from the exact Hessian fits well around the new point x_{k+1} , leading to a better convergence behavior. However, in Figure 2.3c, the standard secant update only matches the gradients at these points, neglecting the function value at x_k . It can severely misestimate the curvature, leading to a poor approximation of the true objective function.

We can overcome this limitation by utilizing the function values. The basic idea is illustrated in Figure 2.4. Even when the gradients are the same at two points x_k and x_{k+1} , natural interpolation of function values differs depending on the function values $f(x_k)$ and $f(x_{k+1})$. This observation motivates the modified secant condition, which incorporates function value information to improve Hessian approximation.

In the following, we present two well-known modified secant conditions.

Function-Value-Based Modified Secant Condition

The first modification incorporates the function value at the previous point into the quadratic model [1, 52, 56]. Let us consider an alternate model with a different approximate Hessian B_{k+1}^F :

$$m_{k+1}^F(x) = f(x_{k+1}) + \nabla f(x_{k+1})^\top (x - x_{k+1}) + \frac{1}{2}(x - x_{k+1})^\top B_{k+1}^F(x - x_{k+1}).$$

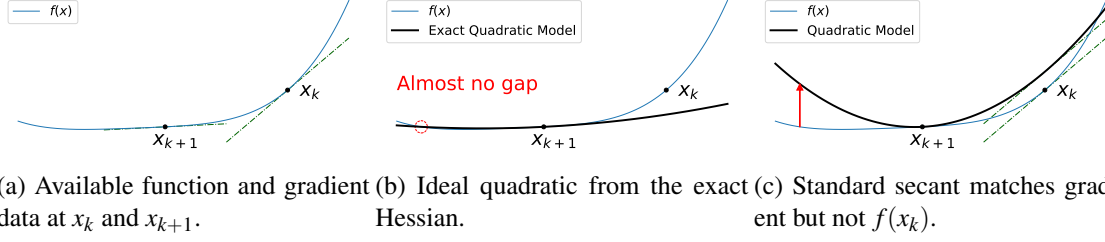


Fig. 2.3: Motivation for modified secant equations. Combining function values with gradients (a) aims to approximate the ideal Newton model (b), while the standard secant update (c) omits $f(x_k)$ and can misestimate curvature.

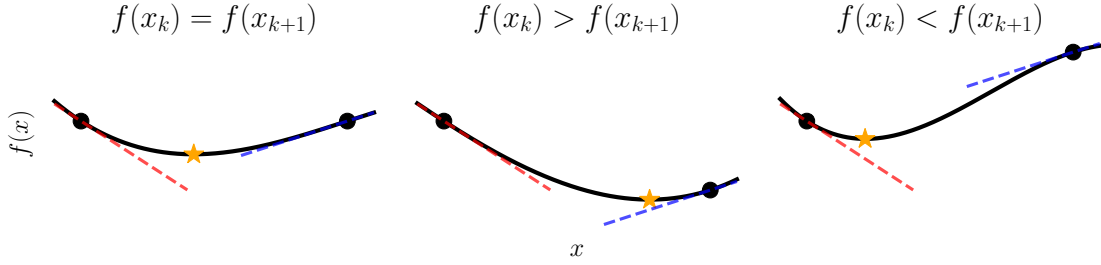


Fig. 2.4: Interpolation with identical gradients at x_k and x_{k+1} but different function values. This leads to distinct interpolants, highlighting the importance of incorporating function value information in Hessian approximation.

We require this model to satisfy

$$m_{k+1}^F(x_k) = f(x_k),$$

which enforces that the function value at the previous point is correctly modeled. This is the key idea behind function-value-based modified secant conditions.

Let us derive the corresponding modified secant condition for B_{k+1}^F . Substituting the definition of m_{k+1}^F and using $s_k = x_{k+1} - x_k$, the condition becomes

$$\begin{aligned} f(x_k) &= f(x_{k+1}) + \nabla f(x_{k+1})^\top (x_k - x_{k+1}) + \frac{1}{2} (x_k - x_{k+1})^\top B_{k+1}^F (x_k - x_{k+1}) \\ &= f(x_{k+1}) - \nabla f(x_{k+1})^\top s_k + \frac{1}{2} s_k^\top B_{k+1}^F s_k. \end{aligned}$$

Now, assume that the updated matrix has the form

$$B_{k+1}^F s_k = y_k - \sigma_k^F s_k,$$

where $\sigma_k^F \in \mathbb{R}$ is a scalar to be determined. Substituting this equation and using $s_k^\top B_{k+1}^F s_k = s_k^\top y_k - \sigma_k^F \|s_k\|^2$ gives

$$f(x_k) = f(x_{k+1}) - \nabla f(x_{k+1})^\top s_k + \frac{1}{2} s_k^\top y_k - \frac{\sigma_k^F}{2} \|s_k\|^2.$$

Solving for σ_k^F and using $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, we obtain

$$\sigma_k^F = \frac{2(f(x_{k+1}) - f(x_k)) - (2\nabla f(x_{k+1}) - y_k)^\top s_k}{\|s_k\|^2}$$

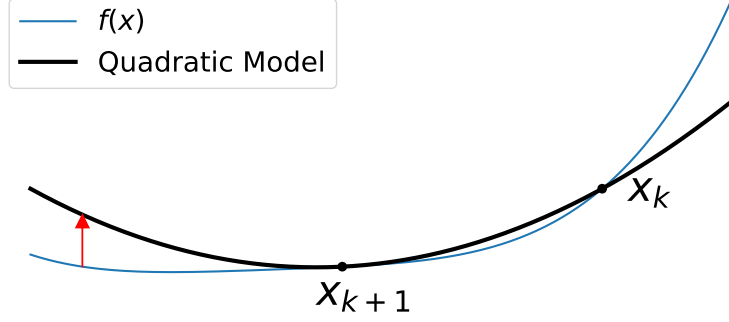


Fig. 2.5: Function-value-based modified secant equation. The modified secant equation $B_k^F s_k = y_k + \sigma_k^F s_k$ constructs a quadratic model that satisfies the function value condition $m_k^F(x_{k-1}) = f(x_{k-1})$ at the previous point. This differs from the standard secant equation, which only matches gradients but not function values.

$$= \frac{2(f(x_{k+1}) - f(x_k)) - (\nabla f(x_{k+1}) + \nabla f(x_k))^T s_k}{\|s_k\|^2}.$$

Therefore, the modified secant condition becomes

$$B_{k+1}^F s_k = y_k + \frac{2(f(x_k) - f(x_{k+1})) + (\nabla f(x_{k+1}) + \nabla f(x_k))^T s_k}{\|s_k\|^2} s_k.$$

To preserve the possibility of maintaining positive definiteness under a BFGS-type update when $s_k^T y_k > 0$, we can modify the formula by taking the maximum with zero in the numerator.

$$B_{k+1}^{F'} s_k = y_k + \frac{\max(0, 2(f(x_k) - f(x_{k+1})) + (\nabla f(x_{k+1}) + \nabla f(x_k))^T s_k)}{\|s_k\|^2} s_k.$$

This is the function-value-based modified secant condition. Unlike the standard secant condition, this formulation ensures that the quadratic model matches not only the function value at the previous point but also the gradients at both points. Refer to Figure 2.5 for an illustration.

Cubic-Augmented Modified Secant Condition

The second modification introduces a cubic term to the model, allowing simultaneous satisfaction of both function value and gradient matching at the previous point [54, 58, 59]. Let $T_{k+1} \in \mathbb{R}^{n \times n \times n}$ be the third-order derivative tensor of f at x_{k+1} such that

$$s_k^T (T_{k+1} s_k) s_k = \sum_{i,j,l=1}^n \partial_{x_i x_j x_l} f(x_{k+1}) s_k^{(i)} s_k^{(j)} s_k^{(l)},$$

where $\partial_{x_i x_j x_l} f$ denotes the third derivative of f with respect to x_i , x_j , and x_l , and $s_k^{(i)}$ is the i -th component of the vector s_k . We introduced this tensor solely for analysis purposes, and it will be eliminated from the final formula.

By incorporating this tensor term, we can define the following cubic-augmented model:

$$\begin{aligned} m_{k+1}^C(x) &= f(x_{k+1}) + \nabla f(x_{k+1})^T (x - x_{k+1}) + \frac{1}{2} (x - x_{k+1})^T B_{k+1}^C (x - x_{k+1}) \\ &\quad + \frac{1}{6} (x - x_{k+1})^T (T_{k+1} (x - x_{k+1})) (x - x_{k+1}). \end{aligned}$$

With this model, we can enforce both function value and gradient matching at the previous point x_k . Specifically, we require

$$\begin{cases} m_{k+1}^C(x_k) = f(x_k) \\ \nabla m_{k+1}^C(x_k) = \nabla f(x_k) \end{cases}$$

By substituting the definition of m_{k+1}^C and using $s_k = x_{k+1} - x_k$, we can rewrite these conditions as

$$\begin{aligned} f(x_k) &= f(x_{k+1}) - s_k^\top \nabla f(x_{k+1}) + \frac{1}{2} s_k^\top B_{k+1}^C s_k - \frac{1}{6} s_k^\top (T_{k+1} s_k) s_k, \\ \nabla f(x_k) &= \nabla f(x_{k+1}) - B_{k+1}^C s_k + \frac{1}{2} (T_{k+1} s_k) s_k. \end{aligned}$$

Rearranging and multiplying both sides by 3 for the first equation, and taking the inner product with s_k for the second equation, we have

$$\begin{aligned} 3(f(x_k) - f(x_{k+1})) + 3s_k^\top \nabla f(x_{k+1}) &= \frac{3}{2} s_k^\top B_{k+1}^C s_k - \frac{1}{2} s_k^\top (T_{k+1} s_k) s_k, \\ -s_k^\top y_k &= -s_k^\top B_{k+1}^C s_k + \frac{1}{2} s_k^\top (T_{k+1} s_k) s_k. \end{aligned}$$

By summing up and using $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, we eliminate the tensor term and obtain the scalar identity

$$3(f(x_k) - f(x_{k+1})) + \frac{3}{2} s_k^\top (\nabla f(x_{k+1}) + \nabla f(x_k)) + \frac{1}{2} s_k^\top y_k = \frac{1}{2} s_k^\top B_{k+1}^C s_k.$$

Assume again that $B_{k+1}^C s_k$ is a linear combination of y_k and s_k , i.e., $B_{k+1}^C s_k = y_k + \sigma_k^C s_k$ for some scalar σ_k^C . Then, the previous equation yields

$$\sigma_k^C = \frac{6(f(x_k) - f(x_{k+1})) + 3s_k^\top (\nabla f(x_k) + \nabla f(x_{k+1}))}{\|s_k\|^2}.$$

Thus, the modified secant condition becomes

$$B_{k+1}^C s_k = y_k + \frac{6(f(x_k) - f(x_{k+1})) + 3s_k^\top (\nabla f(x_k) + \nabla f(x_{k+1}))}{\|s_k\|^2} s_k.$$

This is a cubic-augmented modified secant condition. Unlike the function-value-based quadratic modification, this formulation allows simultaneous satisfaction of both the function value and gradient conditions at the previous point through the introduction of the tensor term. Refer to Figure 2.6 for an illustration.

2.2.3 Other Curvature Storage Methods

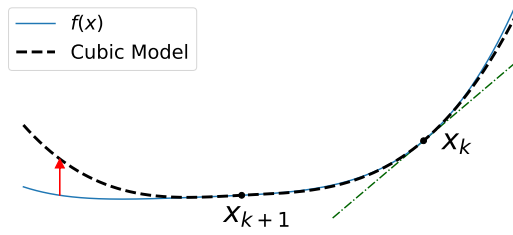
For storing the curvature information, there are several topics. Agg-BFGS [7] is an alternative approach to managing curvature information by aggregating data rather than discarding the oldest information and adding the newest. Multi-Secant [28] extends the secant condition framework by maintaining multiple pairs of step and gradient difference vectors. In the standard formulation, we define

$$s_i = x_{i+1} - x_i, \quad y_i = \nabla f(x_{i+1}) - \nabla f(x_i). \quad (i = k-m, \dots, k)$$

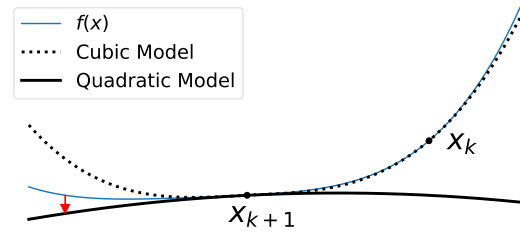
An alternative anchored formulation centers all vectors at the most recent iterate:

$$s_i = x_{k+1} - x_i, \quad y_i = \nabla f(x_{k+1}) - \nabla f(x_i). \quad (i = k-m, \dots, k)$$

This anchored approach provides a different perspective on utilizing historical information for improved Hessian approximation.



(a) Cubic-augmented modified secant.



(b) Quadratic expansion of the cubic-augmented model.

Fig. 2.6: Cubic-augmented modified secant equation. In Figure 2.6a, by incorporating a cubic term $\eta \|x - x_k\|^3$ in the model, the modified secant equation $B_k^C s_k = y_k + \sigma_k^C s_k$ enables simultaneous satisfaction of both conditions at the previous point. In Figure 2.6b, while the cubic model satisfies both function value and gradient conditions, its underlying quadratic component may be indefinite or negative definite.

Chapter 3

Noise-Tolerant Regularized Quasi-Newton Method

In this chapter, we present the proposed regularized quasi-Newton method that is robust against inexact function values. This chapter is of our main contribution.

3.1 Problem Setting and Preliminaries

3.1.1 Problem Setting

We consider optimization problems in which exact objective function values are unavailable, following the recent literature on error-aware optimization [5, 10, 11, 21, 43, 46]. When both objective values and gradients are heavily corrupted by noise, reliable optimization cannot be guaranteed. Thus, meaningful optimization requires that at least one source of information be sufficiently accurate. This work focuses on problems arising from finite-precision computations or inexact evaluations, where objective function evaluations are subject to non-negligible errors while gradient information can be computed with relatively high accuracy. Settings in which objective values are accurate but gradients are noisy, such as derivative-free optimization or stochastic approximation methods [6, 45], are beyond the scope of this work.

In the following, we formalize the assumptions on function and gradient evaluations. We assume that only an error contaminated function $\bar{f}(x)$ is accessible, satisfying the hybrid absolute-relative error model [25, 26]:

$$|f(x) - \bar{f}(x)| \leq \varepsilon_f \max(1, |f(x)|), \quad (3.1)$$

where $0 \leq \varepsilon_f < 1$ is an error rate. This model captures the typical behavior of rounding errors in IEEE 754 floating-point arithmetic [25], and in our numerical experiments, ε_f is chosen to range from 10^2 to 10^7 times the machine epsilon. The error is predominantly relative when $|f(x)|$ is large and effectively absolute when $|f(x)|$ is small. Closely related models are also employed in practical stopping criteria [49].

We next state assumptions regarding gradient evaluations. In practice, termination is typically based on a gradient tolerance $\varepsilon_{\text{gtol}}$, and considers $\|\nabla \bar{f}(x)\| \leq \varepsilon_{\text{gtol}}$ as a stopping criterion. In our numerical experiments, following common implementations [49], we use the infinity norm for the gradient, though other norms could equally be employed. If the gradient noise were comparable to or larger than $\varepsilon_{\text{gtol}}$, the stopping criterion could not be reliably satisfied, no matter how close x is to a stationary point. Therefore, $\varepsilon_{\text{gtol}}$ must be chosen sufficiently large than the gradient noise level. Accordingly, it is natural to assume that the error in gradient evaluations is small compared to the prescribed tolerance:

$$\|\nabla f(x) - \nabla \bar{f}(x)\| \ll \varepsilon_{\text{gtol}}.$$

Since $\|\nabla \bar{f}(x)\| > \varepsilon_{\text{tol}}$ holds prior to termination, the gradient noise is also negligible compared to the gradient norm itself. Consequently, in our theoretical analysis, we assume exact gradients:

$$\|\nabla f(x) - \nabla \bar{f}(x)\| = 0. \quad (3.2)$$

The exactness of $\bar{f}$ in eq. (3.2) serves as a theoretical modeling choice just for simplicity, rather than a claim about practical gradient accuracy. Our numerical experiments in section 3.5 also test the case where gradients are contaminated by noise, demonstrating the practical effectiveness of the proposed method even when eq. (3.2) is violated.

3.1.2 Regularized Quasi-Newton Methods

In classical Newton and quasi-Newton methods, line search techniques are commonly used in practice [49]. An alternative approach is to add regularization terms to the Hessian approximation. Recent years have seen renewed interest in regularized Newton-type methods, including cubic regularization schemes [37, 42, 50]. In this work, we focus on quadratic regularization [13, 27, 30, 44, 47, 48, 57].

Let $g_k := \nabla f(x_k)$ and let $B_k \in \mathbb{R}^{n \times n}$ denote an approximate Hessian. In line-search-based quasi-Newton methods, the update is given by

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_k = -B_k^{-1} g_k,$$

where α_k is determined by a line search. In contrast, regularized quasi-Newton methods compute the step as

$$x_{k+1} = x_k + d_k, \quad d_k = -(B_k + \mu_k I)^{-1} g_k,$$

with a regularization parameter $\mu_k \geq 0$. Combining regularization with line search is also possible and has been explored [47].

A fundamental strategy for choosing μ_k in nonconvex optimization was proposed in earlier work on regularized Newton methods [44, 47, 48]. Their formulation sets

$$\mu_k = c_1 \max(0, -\lambda_{\min}(B_k)) + c_2 \|\nabla f(x_k)\|^\delta, \quad (3.3)$$

where $c_1 > 1$, $c_2 > 0$, $\delta \geq 0$ and $\lambda_{\min}(B_k)$ denotes the smallest eigenvalue of B_k . $\|\cdot\|$ is the Euclidean norm. This choice guarantees that $B_k + \mu_k I$ is positive definite, ensuring that d_k is a descent direction. The regularization strategy adopted in this work is closely related in spirit, and we adapt it to noisy evaluation environments.

3.1.3 Objective-Function-Free Optimization

Objective-Function-Free Optimization (OFFO) refers to a class of methods that avoid explicit evaluations of the objective function and rely solely on gradient information [22, 23]. Such methods are particularly attractive when function values are unreliable due to noise. Similar adaptive trust-region approaches are also discussed [20].

Prominent examples of OFFO include adaptive gradient methods such as AdaGrad [15] and AdaGrad-Norm [51]. Their update rules are

$$\begin{aligned} x_{k+1}^{(i)} &= x_k^{(i)} - \frac{1}{\theta \sqrt{\varsigma + \sum_{j=0}^k (g_j^{(i)})^2}} g_k^{(i)}, & (i = 1, 2, \dots, n) \\ x_{k+1} &= x_k - \frac{1}{\theta \sqrt{\varsigma + \sum_{j=0}^k \|g_j\|^2}} g_k, \end{aligned}$$

respectively, where $\varsigma > 0$ and $\theta > 0$ are algorithmic parameters, and $x^{(i)}$ denotes the i -th component of a vector x .

3.2 Proposed Algorithm

We propose a new algorithm for the problem setting stated in section 3.1.1. We draw inspiration from the OFFO framework to design robust regularization and scaling strategies. Unlike purely objective-function-free methods, our algorithm exploits function values whenever they are numerically reliable, while smoothly transitioning to gradient-based control when they are not. This hybrid behavior enables improved stability without sacrificing efficiency in practical optimization tasks.

The pseudo-code of our algorithm is given in Algorithm 1, where we note that the minimum of the empty set is defined as $+\infty$ in Line 3. Algorithm 2 is an auxiliary line search algorithm called in Algorithm 1.

Algorithm 1: Noise Tolerant Regularized Quasi-Newton Method

Input: $x_0, 0 < k_{\max}, 0 \leq \varepsilon_f < 1, 0 < c < 1, 0 < \theta_{\min} \leq \theta_{\max}$ and $0 < \varsigma < 1$

- 1 $k \leftarrow 0, g_0 \leftarrow \nabla \bar{f}(x_0), K^0 \leftarrow \emptyset, K^+ \leftarrow \emptyset$
- 2 **for** $k = 0, 1, 2, \dots, k_{\max} - 1$ **do**
- 3 **if** $\min_{j \in K^0} (\bar{f}(x_j) - \Delta_j) \geq \bar{f}(x_k)$ **then**
- 4 $K^0 \leftarrow K^0 \cup \{k\}$
- 5 $\mu_k \leftarrow 0$
- 6 **else**
- 7 $K^+ \leftarrow K^+ \cup \{k\}$
- 8 $\mu_k \leftarrow \theta_k \sqrt{\varsigma + \sum_{j \in K^+} \|g_j\|^2}$ with $\theta_k \in [\theta_{\min}, \theta_{\max}]$ chosen adaptively.
- 9 $d_k \leftarrow -(B_k + \mu_k I)^{-1} g_k$ with B_k chosen properly (section 3.4.1).
- 10 Find α_k and Δ_k satisfying eq. (3.4) by Algorithm 2.
- 11 $x_{k+1} \leftarrow x_k + \alpha_k d_k, g_{k+1} \leftarrow \nabla \bar{f}(x_{k+1})$
- 12 **end**

Algorithm 2: Backtracking Line Search with Relaxed Armijo Condition

Input: $x_k \in \mathbb{R}^n, d_k \in \mathbb{R}^n, g_k \in \mathbb{R}^n, 0 < c < 1$ and $0 < \beta_{\min} < \beta_{\max} < 1$

- 1 $\alpha_k \leftarrow 1$
- 2 $\Delta_k \leftarrow \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k))$
- 3 **while** $\bar{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k < \bar{f}(x_k + \alpha_k d_k)$ **do**
- 4 $\alpha_k \leftarrow \beta \alpha_k$ with $\beta \in [\beta_{\min}, \beta_{\max}]$ chosen adaptively (section 3.4.2).
- 5 $\Delta_k \leftarrow \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k))$
- 6 **end**
- 7 **return** α_k

In the following, we explain the three main components of Algorithm 1.

Computation of the Regularized Quasi-Newton Direction (Line 9): The first component is the computation of the regularized quasi-Newton direction d_k . Given $g_k = \nabla \bar{f}(x_k) = \nabla f(x_k)$, an approximate Hessian B_k , and a regularization parameter μ_k , we compute the search direction $d_k = -(B_k + \mu_k I)^{-1} g_k$ as in section 3.1.2. For the theoretical analysis, any choice of B_k satisfy-

ing Assumption 3 stated in section 3.3.1 is admissible, which ensures B_k is positive definite. A practical choice of B_k is discussed in section 3.4.1.

Relaxed Armijo Condition with an Error-Absorbing Term (Line 10): The second component is the computation of the step size α_k using a relaxed Armijo condition. We compute α_k satisfying the relaxed Armijo condition

$$\begin{cases} \bar{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k \geq \bar{f}(x_k + \alpha_k d_k), \\ \Delta_k = \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k)) \end{cases} \quad (3.4)$$

where Δ_k is the error-absorbing term that is recomputed for each α_k . This relaxation guarantees the existence of $\alpha_k \in (0, 1]$ even in the presence of numerical errors in $\bar{f}$. When d_k is a descent direction ($g_k^\top d_k \leq 0$), eq. (3.4) ensures a temporary increase of $\bar{f}$ is at most Δ_k . Since Δ_k depends on $-\bar{f}(x_k + \alpha_k d_k)$ but not on $\bar{f}(x_k + \alpha_k d_k)$, the value of Δ_k is independent of how much $\bar{f}(x_k + \alpha_k d_k)$ increases, preventing unbounded growth of Δ_k .

Adaptive Update of the Regularization Parameter (Lines 3–8): The third component is the adaptive update of the regularization parameter μ_k . A larger μ_k yields a more conservative step, while a smaller μ_k allows a more aggressive quasi-Newton step, especially when $\mu_k = 0$. In practical settings, setting $\mu_k = 0$ is preferred, since this allows us to fully exploit the quasi-Newton approximation and typically leads to practically faster convergence. Let us partition the iteration indices $K := \{0, 1, 2, \dots, k_{\max} - 1\}$ into two sets:

$$K^0 := \{k \in K \mid \mu_k = 0\}, \quad K^+ := \{k \in K \mid \mu_k > 0\}. \quad (3.5)$$

We set $\mu_k = 0$ if and only if the following condition is satisfied:

$$\min_{j \in K^0, j < k} (\bar{f}(x_j) - \Delta_j) \geq \bar{f}(x_k) \quad (3.6)$$

Note that $k = 0$ satisfies eq. (3.6) since K^0 is empty. This condition restricts $\mu_k = 0$ to iterations for which a sufficient decrease in $\bar{f}$ has been observed. Thereby, while $\bar{f}$ can temporarily increase, this condition theoretically ensures that $\bar{f}$ does not grow unboundedly or oscillate indefinitely. See ?? for an illustrative example.

Otherwise, to ensure convergence to first-order stationary points, we apply an OFFO-based update

$$\mu_k = \theta_k \sqrt{\varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2}, \quad (3.7)$$

as described in section 3.1.3. Since OFFO schemes do not rely on function values, this update is robust to numerical errors in $\bar{f}$.

3.3 Theoretical Analysis

In this section, we establish the global convergence properties of Algorithm 1 under the problem setting in section 3.1.1, namely, eqs. (3.1) and (3.2).

3.3.1 Assumptions

Throughout this section, we make the following three assumptions. The first assumption guarantees that the objective function is bounded below.

Assumption 1. The function f is bounded below; that is, for all $x \in \mathbb{R}^n$,

$$f(x) \geq f^* := \inf_{x \in \mathbb{R}^n} f(x) > -\infty.$$

By the triangle inequality and eq. (3.1), we have $\bar{f}(x) \geq f(x) - \varepsilon_f \max(1, |f(x)|)$. Since the function $g(t) = t - \varepsilon_f \max(1, |t|)$ is non-decreasing for $0 \leq \varepsilon_f < 1$, it follows that $\inf_x g(f(x)) = g(f^*)$. Consequently, we can also lower bound $\bar{f}$ as

$$\bar{f}(x) \geq \bar{f}^* := \inf_{x \in \mathbb{R}^n} (f(x) - \varepsilon_f \max(1, |f(x)|)) = f^* - \varepsilon_f \max(1, |f^*|) > -\infty. \quad (3.8)$$

The second assumption ensures smoothness of the objective function.

Assumption 2. The function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is L -smooth, meaning that there exists a constant $L > 0$ such that for all $x, y \in \mathbb{R}^n$,

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|y - x\|^2.$$

The third assumption requires the sequence of matrices $\{B_k\}$ to be uniformly bounded and uniformly positive definite.

Assumption 3. There exist constants $0 < m \leq M$ such that all B_k in Algorithm 1 satisfy

$$mI \preceq B_k \preceq MI.$$

As we will discuss in section 3.4, the standard BFGS update with a slight modification ensures that Assumption 3 holds in practice. By Assumption 3, we can derive two useful bounds involving g_k and d_k . Using $d_k = -(B_k + \mu_k I)^{-1} g_k$ (Line 9) and Assumption 3 asserting $(m + \mu_k)I \preceq B_k + \mu_k I \preceq (M + \mu_k)I$, we obtain the following two bounds:

$$-g_k^\top d_k = g_k^\top (B_k + \mu_k I)^{-1} g_k \geq \frac{1}{M + \mu_k} \|g_k\|^2, \quad (3.9)$$

$$\|d_k\|^2 = \|(B_k + \mu_k I)^{-1} g_k\|^2 \leq \frac{1}{(m + \mu_k)^2} \|g_k\|^2. \quad (3.10)$$

3.3.2 Error Bound on Function Evaluations

In the following, we define the actual and observed function decreases as

$$\delta_k := f(x_k) - f(x_{k+1}), \quad (3.11)$$

$$\bar{\delta}_k := \bar{f}(x_k) - \bar{f}(x_{k+1}). \quad (3.12)$$

We first prove a useful property of Δ_k defined in eq. (3.4). The following lemma asserts that the error between the actual values difference δ_k and the observed values difference $\bar{\delta}_k$ can be bounded by Δ_k .

Lemma 1. If x_k and x_{k+1} satisfies $f(x_{k+1}) \leq f(x_k)$, it holds that

$$\delta_k \leq \bar{\delta}_k + \Delta_k. \quad (3.13)$$

Proof. We start from a general observation. We claim that, for all $x \in \mathbb{R}^n$,

$$\max(1, \pm f(x)) \leq \frac{1}{1 - \varepsilon_f} \max(1, \pm \bar{f}(x)). \quad (3.14)$$

The proof of eq. (3.14) is as follows. We only consider the case with the plus sign, as the other case can be treated analogously. If $f(x) \leq 1$, then $\max(1, f(x)) = 1$ and the inequality is trivial. If $f(x) > 1$, eq. (3.1) implies

$$|f(x) - \bar{f}(x)| \leq \varepsilon_f \max(1, |f(x)|) = \varepsilon_f f(x). \quad (3.15)$$

Since $\varepsilon_f < 1$, eq. (3.15) implies that $f(x)$ and $\bar{f}(x)$ have the same sign. Moreover, by the triangle inequality, $|f(x)| - |\bar{f}(x)| \leq \varepsilon_f |f(x)|$, which yields $\max(1, f(x)) = f(x) \leq \frac{1}{1-\varepsilon_f} \bar{f}(x)$. Combining the two cases completes the proof of eq. (3.14).

With these preparations, we now prove the lemma. For all $k \in K$, we have

$$\begin{aligned} |\delta_k - \bar{\delta}_k| &= |(f(x_k) - \bar{f}(x_k)) - (f(x_{k+1}) - \bar{f}(x_{k+1}))| && \text{(rearrangement)} \\ &\leq \varepsilon_f \max(1, |f(x_k)|) + \varepsilon_f \max(1, |f(x_{k+1})|) && \text{(definition in eq. (3.1))} \\ &\leq 2\varepsilon_f \max(1, |f(x_k)|, |f(x_{k+1})|) && \text{(max property)} \end{aligned} \quad (3.16)$$

From the condition of this lemma ($f(x_{k+1}) \leq f(x_k)$), this can be further bounded as

$$\begin{aligned} 2\varepsilon_f \max(1, |f(x_k)|, |f(x_{k+1})|) &= 2\varepsilon_f \max(1, f(x_k), -f(x_{k+1})) && (f(x_{k+1}) \leq f(x_k)) \\ &\leq \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_{k+1})) && \text{(eq. (3.14))} \\ &= \Delta_k, && \text{(eq. (3.4))} \end{aligned}$$

which yields the desired inequality by the triangle inequality. $\square$

We also derive the following lemma for later use.

Lemma 2. *If x_k and x_{k+1} satisfy the relaxed Armijo condition (eq. (3.4)) with a descent direction d_k (i.e., $g_k^\top d_k \leq 0$), it holds that*

$$\bar{\delta}_k \leq \delta_k + \frac{1+\varepsilon_f}{1-\varepsilon_f} \Delta_k.$$

Proof. By condition of this lemma and eq. (3.4), we have

$$\bar{f}(x_k) - \bar{f}(x_{k+1}) \geq -c\alpha_k g_k^\top d_k - \Delta_k \geq -\Delta_k. \quad (3.17)$$

Using eqs. (3.14) and (3.17), the right-hand side of eq. (3.16) can be further bounded as

$$\begin{aligned} 2\varepsilon_f \max(1, |f(x_k)|, |f(x_{k+1})|) &\leq \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k), \bar{f}(x_{k+1}), -\bar{f}(x_{k+1})) \\ &\leq \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k) + \Delta_k, -\bar{f}(x_{k+1}) + \Delta_k) \\ &\leq \frac{2\varepsilon_f}{1-\varepsilon_f} (\max(1, \bar{f}(x_k), -\bar{f}(x_{k+1})) + \Delta_k). \end{aligned}$$

Combining this with eq. (3.16) and the definition of Δ_k in eq. (3.4) yields

$$|\delta_k - \bar{\delta}_k| \leq \frac{2\varepsilon_f}{1-\varepsilon_f} (\max(1, \bar{f}(x_k), -\bar{f}(x_{k+1})) + \Delta_k) = \left(1 + \frac{2\varepsilon_f}{1-\varepsilon_f}\right) \Delta_k = \frac{1+\varepsilon_f}{1-\varepsilon_f} \Delta_k.$$

Applying the triangle inequality completes the proof. $\square$

3.3.3 Backtracking Stage

We next analyze the line search in Algorithm 2. Recall that we assume that the parameters in Algorithm 2 satisfy $0 < c < 1$ and $0 < \beta_{\min} < \beta_{\max} < 1$.

Proposition 1. *Under Assumptions 2 and 3, the backtracking procedure in Algorithm 2 terminates whenever the step size α_k satisfies*

$$\alpha_k \leq \frac{2(1-c)m}{L}. \quad (3.18)$$

Proof. Under the L -smoothness of f (Assumption 2), we always have

$$\begin{aligned} f(x_k) - f(x_k + \alpha_k d_k) &\geq -\alpha_k g_k^\top d_k - \frac{L}{2} \alpha_k^2 \|d_k\|^2 \\ &= -c\alpha_k g_k^\top d_k + \alpha_k \left(-(1-c)g_k^\top d_k - \frac{L\alpha_k}{2} \|d_k\|^2 \right). \end{aligned} \quad (3.19)$$

From Line 9 of Algorithm 1, it follows that $g_k = -(B_k + \mu_k I)d_k$. Then, from Assumption 3 and $\mu_k \geq 0$, we have

$$-g_k^\top d_k = d_k^\top (B_k + \mu_k I)d_k \geq (m + \mu_k) \|d_k\|^2 \geq 0. \quad (3.20)$$

Thus, when α_k satisfies the bound in eq. (3.18), we have

$$-(1-c)g_k^\top d_k - \frac{L\alpha_k}{2} \|d_k\|^2 \geq \left((1-c)m - \frac{L\alpha_k}{2} \right) \|d_k\|^2 \geq 0. \quad (3.21)$$

Combining eqs. (3.19) and (3.21) yields

$$f(x_k) - f(x_k + \alpha_k d_k) \geq -c\alpha_k g_k^\top d_k \geq 0.$$

Hence, the condition of Lemma 1 is satisfied with $x_{k+1} = x_k + \alpha_k d_k$ and thus we obtain $\delta_k \leq \bar{\delta}_k + \Delta_k$, i.e.,

$$-c\alpha_k g_k^\top d_k \leq f(x_k) - f(x_k + \alpha_k d_k) \leq \bar{f}(x_k) - \bar{f}(x_k + \alpha_k d_k) + \Delta_k.$$

This inequality means that the relaxed Armijo condition (eq. (3.4)) is satisfied and thus the backtracking procedure terminates. This completes the proof. $\square$

Now, let us analyze the behavior of Algorithm 2 using Proposition 1. Define

$$\bar{\alpha} := \beta_{\min} \min \left(\frac{2(1-c)m}{L}, 1 \right) > 0. \quad (3.22)$$

The following corollary summarizes two immediate consequences: the backtracking procedure terminates after finitely many rejections, and the resulting step size is uniformly bounded by $\bar{\alpha}$.

Corollary 1. *Under Assumptions 2 and 3, the backtracking line search in Algorithm 2 terminates after finitely many rejections, and the step size α_k satisfies*

$$\alpha_k \geq \bar{\alpha}.$$

Proof. Let r denote the number of backtracking rejections. Initially, $\alpha_k = 1$, and each rejection multiplies α_k by $\beta \leq \beta_{\max} < 1$. Hence, after r rejections, $\alpha_k \leq \beta_{\max}^r$ holds. By Proposition 1, if

$$\alpha_k \leq \beta_{\max}^r \leq \frac{2(1-c)m}{L}, \quad (3.23)$$

the backtracking procedure must terminate. Since $0 < c < 1$ and $0 < m$, the right-hand side is positive, and thus r is finite. We next establish the lower bound of α_k . For the case $r = 0$, $\alpha_k = 1 \geq \bar{\alpha}$ by $\beta_{\min} < 1$. Otherwise, let α'_k denote the last rejected step size. By Proposition 1, we have

$$\alpha'_k > \frac{2(1-c)m}{L} \geq \min\left(\frac{2(1-c)m}{L}, 1\right). \quad (3.24)$$

The accepted step size α_k is chosen from the interval $[\beta_{\min}\alpha'_k, \beta_{\max}\alpha'_k]$, and thus

$$\alpha_k \geq \beta_{\min}\alpha'_k > \beta_{\min} \min\left(\frac{2(1-c)m}{L}, 1\right) = \bar{\alpha}. \quad (3.25)$$

This completes the proof. $\square$

3.3.4 Bound for the case $\mu_k = 0$

In this subsection, we bound the sum of $\|g_k\|^2$ over $k \in K^0$. Recall that K^0 be a set of iteration indices k such that $\mu_k = 0$ in Algorithm 1 as defined in eq. (3.5).

To this end, we first establish that the sum of Δ_k over $k \in K^0$ is bounded, where Δ_k is the error-absorbing term in the relaxed Armijo condition (eq. (3.4)). To this purpose, we introduce the following constant:

$$\Delta_{\max} := \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_0), -\bar{f}^*). \quad (3.26)$$

Lemma 3. *Under Assumptions 1 to 3, the sum of Δ_k over $k \in K^0$ satisfies*

$$\sum_{k \in K^0} \Delta_k \leq \bar{f}(x_0) - \bar{f}^* + \Delta_{\max}.$$

Proof. Let us define the last index of $|K^0|$ as ℓ , and enumerate the elements of K^0 in increasing order as $K^0 = (k_i)_{i=0}^{\ell}$. For all $i < \ell$, using eq. (3.6) with $j = k_i$ and $k = k_{i+1}$, we obtain

$$\Delta_{k_i} \leq \bar{f}(x_{k_i}) - \bar{f}(x_{k_{i+1}}). \quad (3.27)$$

Summing up eq. (3.27) over $i < \ell$ and adding the case k_{ℓ} yields

$$\sum_{k \in K^0} \Delta_k \leq \sum_{i=0}^{\ell-1} (\bar{f}(x_{k_i}) - \bar{f}(x_{k_{i+1}})) + \Delta_{k_{\ell}} = \bar{f}(x_0) - \bar{f}(x_{k_{\ell}}) + \Delta_{k_{\ell}}. \quad (3.28)$$

It also holds that $\bar{f}(x_0) \geq \bar{f}(x_{k_{\ell}})$ from eq. (3.6) with $j = 0$ and $k = k_{\ell}$. We also have $\bar{f}(x_{k_{\ell}}) \geq \bar{f}^*$ from Assumption 1 and eq. (3.8). Thus, eq. (3.28) can be further bounded as

$$\sum_{k \in K^0} \Delta_k \leq \bar{f}(x_0) - \bar{f}(x_{k_{\ell}}) + \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_{k_{\ell}}), -\bar{f}(x_{k_{\ell}+1})) \quad (\text{eq. (3.4)})$$

$$\leq \bar{f}(x_0) - \bar{f}^* + \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_0), -\bar{f}^*) \quad (\text{eq. (3.8)})$$

$$= \bar{f}(x_0) - \bar{f}^* + \Delta_{\max}. \quad (\text{eq. (3.26)})$$

This completes the proof. $\square$

Using this lemma, we can bound the sum of $\|g_k\|^2$ over $k \in K^0$ using $\delta_k = f(x_k) - f(x_{k+1})$ in eq. (3.11) as follows.

Lemma 4. Under Assumptions 1 to 3, the sum of δ_k over $k \in K^0$ satisfies

$$\sum_{k \in K^0} \delta_k \geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 - \frac{2}{1 - \varepsilon_f} \left(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max} \right).$$

Proof. The relaxed Armijo condition (eq. (3.4)) at iteration $k \in K^0$ yields

$$\begin{aligned} \bar{\delta}_k + \Delta_k &= \bar{f}(x_k) - \bar{f}(x_{k+1}) + \Delta_k && \text{(eq. (3.12))} \\ &\geq -c\alpha_k g_k^\top d_k && \text{(eq. (3.4))} \\ &\geq \frac{c\bar{\alpha}}{M + \mu_k} \|g_k\|^2 && \text{(Corollary 1 and eq. (3.9))} \\ &= \frac{c\bar{\alpha}}{M} \|g_k\|^2. && (\mu_k = 0 \text{ since } k \in K^0) \end{aligned} \quad (3.29)$$

Since the conditions of Lemma 2 are always satisfied, we also have

$$\bar{\delta}_k + \Delta_k \leq \delta_k + \frac{1 + \varepsilon_f}{1 - \varepsilon_f} \Delta_k + \Delta_k = \delta_k + \frac{2}{1 - \varepsilon_f} \Delta_k. \quad (3.30)$$

Then, combining eqs. (3.29) and (3.30) and summing up over $k \in K^0$ gives

$$\sum_{k \in K^0} \frac{c\bar{\alpha}}{M} \|g_k\|^2 \leq \sum_{k \in K^0} \delta_k + \frac{2}{1 - \varepsilon_f} \sum_{k \in K^0} \Delta_k \leq \sum_{k \in K^0} \delta_k + \frac{2}{1 - \varepsilon_f} \left(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max} \right),$$

where the last inequality follows from Lemma 3. This completes the proof. $\square$

3.3.5 Bound for the case $\mu_k > 0$

Next, we show that the sum of $\|g_k\|^2$ over $k \in K^+$ is bounded. Recall that K^+ be a set of iteration indices k such that $\mu_k > 0$ as defined in eq. (3.5), and that μ_k is defined as $\theta_k \sqrt{\varsigma + \sum_{j \in K^+, j < k} \|g_j\|^2}$ with $\theta_k \in [\theta_{\min}, \theta_{\max}]$ in Line 8 of Algorithm 1.

Before proceeding to the main proof, we present standard inequalities in the AdaGrad-Norm algorithm analysis [51, Lemma 3.2], [23, Lemma 3.1]. Its proof is deferred to section A.1.

Lemma 5. In Algorithm 1, we have

$$\begin{aligned} \sum_{k \in K^+} \frac{\|g_k\|^2}{\mu_k^2} &\leq \frac{1}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) - \ln(\varsigma) \right), \\ \sum_{k \in K^+} \frac{\|g_k\|^2}{\mu_k} &\geq \frac{1}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - \sqrt{\varsigma} \right). \end{aligned}$$

We bound the sum of $\|g_k\|^2$ over $k \in K^+$ using δ_k defined in eq. (3.11). Now, we define the following constants for later use:

$$C_1 := \frac{\bar{\alpha}}{\theta_{\max}} \quad C_2 := \frac{M\bar{\alpha} + L/2}{\theta_{\min}^2} \quad (3.31)$$

Lemma 6. Under Assumptions 2 and 3, the sum of δ_k over $k \in K^+$ satisfies

$$\sum_{k \in K^+} \delta_k \geq C_1 \left(\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - \sqrt{\varsigma} \right) - C_2 \left(\ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) - \ln(\varsigma) \right)$$

Proof. From the L -smoothness (Assumption 2), for $k \in K^+$ we have

$$\begin{aligned}
\delta_k &= f(x_k) - f(x_k + \alpha_k d_k) && \text{(eq. (3.11) and } x_{k+1} = x_k + \alpha_k d_k) \\
&\geq -\alpha_k g_k^\top d_k - \frac{\alpha_k^2 L}{2} \|d_k\|^2 && \text{(Assumption 2)} \\
&\geq \left(\frac{\alpha_k}{M + \mu_k} - \frac{\alpha_k^2 L}{2(m + \mu_k)^2} \right) \|g_k\|^2 && \text{(eqs. (3.9) and (3.10))} \\
&\geq \left(\frac{\bar{\alpha}}{M + \mu_k} - \frac{L}{2(m + \mu_k)^2} \right) \|g_k\|^2. && \text{(Corollary 1 and } \alpha_k \leq 1)
\end{aligned} \tag{3.32}$$

We can further bound the terms in eq. (3.32) as

$$\frac{\bar{\alpha}}{M + \mu_k} = \frac{\bar{\alpha}}{\mu_k} \frac{1}{1 + \frac{M}{\mu_k}} \geq \frac{\bar{\alpha}}{\mu_k} \left(1 - \frac{M}{\mu_k} \right) = \frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha}}{\mu_k^2}, \quad \frac{L}{2(m + \mu_k)^2} \leq \frac{L}{2\mu_k^2}. \tag{3.33}$$

Thus, applying eq. (3.33) to eq. (3.32) yields

$$\delta_k \geq \left(\frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha} + L/2}{\mu_k^2} \right) \|g_k\|^2. \tag{3.34}$$

Summing up eq. (3.34) over $k \in K^+$ and applying Lemma 5 yield

$$\begin{aligned}
&\sum_{k \in K^+} \delta_k \\
&\geq \sum_{k \in K^+} \left(\frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha} + L/2}{\mu_k^2} \right) \|g_k\|^2 \\
&\geq \frac{\bar{\alpha}}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - \sqrt{\varsigma} \right) - \frac{M\bar{\alpha} + L/2}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) - \ln(\varsigma) \right),
\end{aligned}$$

which yields the desired result. $\square$

3.3.6 Global Convergence Result

Finally, we combine the results for $k \in K^0$ (Lemma 4) and $k \in K^+$ (Lemma 6) to derive a bound on the total sum of $\|g_k\|^2$. We define the constant F as follows:

$$F := f(x_0) - f^* + \frac{2}{1 - \epsilon_f} \left(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max} \right) + C_1 \sqrt{\varsigma} - C_2 \ln(\varsigma). \tag{3.35}$$

Then, we can obtain the following unified bound of the sum of $\|g_k\|^2$.

Lemma 7. *Under the assumptions, the sum of $\|g_k\|^2$ over $k \in K^0, K^+$ satisfies*

$$F \geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + C_1 \sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - C_2 \ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right),$$

Proof. Since $f(x_{k_{\max}}) \geq f^*$ by Assumption 1 and $K = K^0 \cup K^+$, we have

$$f(x_0) - f^* \geq f(x_0) - f(x_{k_{\max}}) = \sum_{k \in K} \delta_k = \sum_{k \in K^0} \delta_k + \sum_{k \in K^+} \delta_k.$$

From Lemmas 4 and 6, we can further bound as

$$\begin{aligned} f(x_0) - f^* &\geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 - \frac{2}{1 - \varepsilon_f} \left(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max} \right) \\ &\quad + C_1 \sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - C_1 \sqrt{\varsigma} - C_2 \ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) + C_2 \ln(\varsigma). \end{aligned}$$

Substituting eq. (3.35) yields the desired bound, concluding the proof. $\square$

Now, we establish the following global convergence result for Algorithm 1.

Theorem 1 (Global Convergence). *Under Assumptions 1 to 3, the average of squared gradient norms generated by Algorithm 1 satisfies:*

$$\frac{1}{k_{\max}} \sum_{k=0}^{k_{\max}-1} \|g_k\|^2 = \mathcal{O} \left(\frac{1}{k_{\max}} \right).$$

Proof. Let us define the function $\phi: \mathbb{R}_{>0} \rightarrow \mathbb{R}$ as

$$\phi(G) := \frac{C_1}{2} \sqrt{G} - C_2 \ln(G). \quad (3.36)$$

Since $\phi'(G) = \frac{C_1}{4\sqrt{G}} - \frac{C_2}{G}$, its minimum over $G > 0$ is attained at $G^* := \left(\frac{4C_2}{C_1} \right)^2$ with

$$\phi(G^*) = 2C_2 \left(1 - \ln \left(\frac{4C_2}{C_1} \right) \right) = 2C_2 \ln \left(\frac{eC_1}{4C_2} \right),$$

where e is the base of the natural logarithm. Thus, Lemma 7 yields

$$\begin{aligned} F &\geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} + \phi \left(\sum_{k \in K^+} \|g_k\|^2 + \varsigma \right) \\ &\geq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} + \phi(G^*), \end{aligned}$$

which is equivalent to

$$0 \leq \frac{c\bar{\alpha}}{M} \sum_{k \in K^0} \|g_k\|^2 + \frac{C_1}{2} \sqrt{\sum_{k \in K^+} \|g_k\|^2} \leq F' \quad (3.37)$$

where

$$\begin{aligned} F' &:= F - \phi(G^*) \\ &= f(x_0) - f^* + \frac{2}{1 - \varepsilon_f} \left(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max} \right) + C_1 \sqrt{\varsigma} - C_2 \ln(\varsigma) - 2C_2 \ln \left(\frac{eC_1}{4C_2} \right). \end{aligned}$$

As a general fact, we consider the maximization of $x + y$ under the constraint $ax + b\sqrt{y} \leq c$ for $a, b > 0, c \geq 0$ and $x, y \geq 0$. Since $x + y$ is increasing in both variables, we can assume $ax + b\sqrt{y} = c$. On this boundary, we can write $y = \left(\frac{c - ax}{b} \right)^2$, and thus $x + y$ is convex on an interval $0 \leq x \leq c/a$, attaining its maximum at an endpoint. Therefore the maximum of $x + y$ is achieved either at $(x, y) = (c/a, 0)$ or at $(x, y) = (0, (c/b)^2)$. Using this fact with eq. (3.37), we have

$$\frac{1}{k_{\max}} \sum_{k=0}^{k_{\max}-1} \|g_k\|^2 = \frac{1}{k_{\max}} \left(\sum_{k \in K^0} \|g_k\|^2 + \sum_{k \in K^+} \|g_k\|^2 \right) \leq \frac{1}{k_{\max}} \max \left(\frac{M}{c\bar{\alpha}} F', \frac{4}{C_1^2} F'^2 \right),$$

which completes the proof. $\square$

As a remark, the function $\phi(G)$ in eq. (3.36) is closely related to the Lambert W function [12, 23]. When $x \in [-1/e, 0)$, $ye^y = x$ has two real and negative solutions, and the smaller solution is given by the second branch of Lambert W function $y = W_{-1}(x) < 0$. We can explicitly solve the equation $\phi(G) = 0$ using this function, and then we can obtain a tighter bound in Theorem 1. Since this is not essential for the main discussion, we omit the details here. See [23] for more details.

It is well-known that Theorem 1 implies that the iteration complexity to achieve

$$\min \{k \mid \|\nabla f(x_k)\| \leq \varepsilon_{\text{gtol}}\} = \mathcal{O}(1/\varepsilon_{\text{gtol}}^2).$$

This is a standard convergence guarantee for first-order optimization algorithms applied to L -smooth nonconvex functions for such problems. We interpret these results in light of classical optimization results.

- As for the term from K^0 , this is an analogue of the classical gradient descent bound for an error-contaminated function $\bar{f}$. Indeed, for gradient descent with fixed step size $\alpha = 1/L$, the sum of squared gradients is bounded as

$$\sum_{j=0}^k \|g_j\|^2 \leq 2L(f(x_0) - f^*) = \frac{2(f(x_0) - f^*)}{\alpha}.$$

- As for the term from K^+ , this is analogous to the convergence property of AdaGrad-Norm, which scales as $(f(x_0) - f^*)^2$ [51]. This follows from the fact that F' contains the initial optimality gaps $f(x_0) - f^*$ and $\bar{f}(x_0) - \bar{f}^*$, and its term is quadratic in F' .

3.4 Practical Choices of Algorithmic Variables

In sections 3.2 and 3.3, we introduced Algorithm 1 and established its convergence properties. In this section, we discuss practical choices of algorithmic variables and parameters that lead to fast and stable performance in practice. The quantities to be discussed here are the matrices B_k , the step sizes α_k , and the regularization parameters μ_k .

3.4.1 Approximate Hessian B_k

We first discuss the construction of the matrices B_k used in Line 9 of Algorithm 1. As mentioned in section 1.1, we employ an L-BFGS method to form B_k . The purpose of this subsection is to show that B_k can satisfy Assumption 3, the uniform spectral bound, with the modification and selection of curvature pairs.

Construction of B_k : Before stating concrete construction of B_k , we recall the L-BFGS method and the curvature pair notation [7, 31, 38]. For an iteration k , we define

$$s_k := x_{k+1} - x_k, \quad y_k := g_{k+1} - g_k \quad (3.38)$$

where g_k and g_{k+1} are the gradients at x_k and x_{k+1} , respectively. Starting from an initial matrix, an approximate Hessian is obtained by successively applying BFGS updates to a sequence of curvature pairs. Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a collection of curvature pairs, where k_i denotes the iteration index at which the i -th pair was generated. Starting from the initial matrix γI with $\gamma = \|y_{k_0}\|^2 / y_{k_0}^\top s_{k_0}$, we define $\text{BFGS}(\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1})$ as the matrix obtained by sequentially applying the BFGS update using these pairs in the given order. Here, p denotes the maximum number of stored curvature pairs and is a fixed small integer in the L-BFGS method. When fewer than p curvature pairs are available, the BFGS update is applied using all currently available pairs.

Now, we provide a sufficient condition for Assumption 3 by the following proposition, which is a variant of existing work [17, Theorem 5.5] [19, Lemma 1]. Its proof is deferred to section A.2.

Proposition 2. *Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a fixed number of curvature pairs with $s_{k_i} \neq 0$. Assume that there exist constants $0 < \lambda \leq \Lambda$ such that for all $(s, y) \in \{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$,*

$$y^\top s \geq \frac{1}{\Lambda} \|y\|^2, \quad (3.39)$$

$$y^\top s \geq \lambda \|s\|^2. \quad (3.40)$$

Define the condition number $\kappa := \Lambda/\lambda$. Then there exist $0 < m < M$ such that

$$mI \preceq \text{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1}) \preceq MI, \quad (3.41)$$

where

$$M := \gamma + p\Lambda, \quad m := \left((1 + \sqrt{\kappa})^{2p} \left(\gamma + \frac{1}{\lambda(2\sqrt{\kappa} + \kappa)} \right) \right)^{-1}.$$

Proposition 2 shows that, whenever eqs. (3.39) and (3.40) hold for the stored pairs, the resulting BFGS matrix satisfies Assumption 3 as expressed in eq. (3.41).

Based on Proposition 2, we basically adopt the following rule for forming B_k :

$$B_k = \text{BFGS}(\{(s_{k_i}, \bar{y}_{k_i})\}), \quad (3.42)$$

where

$$\bar{y}_{k_i} := \theta_i^{\text{damp}} y_{k_i} + (1 - \theta_i^{\text{damp}}) B_{k-1} s_{k_i}$$

with $\theta_i^{\text{damp}} \in [0, 1]$. This parameter is determined using damped BFGS update [32, 41] and the indices $\{k_i\}_{i=0}^{p-1}$ are selected so that the modified pairs $(s_{k_i}, \bar{y}_{k_i})$ satisfy eqs. (3.39) and (3.40) for some constants λ, Λ . The damped BFGS is a classical technique to ensure $\bar{y}_{k_i}^\top s_{k_i} > 0$, and we adopted its simple limited-memory variant [32].

Since we do not employ Wolfe conditions in the line search, this damping technique plays a central role in ensuring the positive definiteness of B_k . The selection of curvature pairs shares the same spirit as the cautious update [29, 33, 34], which only uses pairs satisfying certain curvature conditions. By Proposition 2, B_k satisfies the uniform spectral bound in Assumption 3. This confirms that the practical construction of B_k is consistent with the theoretical requirements.

Remarks for Practical Implementation: Having established that the proposed construction of B_k satisfies Assumption 3, we now present several remarks on its practical implementation. We begin by describing how to compute the search direction

$$d_k = -(B_k + \mu_k I)^{-1} g_k \quad (3.43)$$

with B_k in eq. (3.42). When $\mu_k = 0$, the direction can be computed efficiently by the standard L-BFGS two-loop recursion [31, 38]. For the case $\mu_k > 0$, regularized limited-memory schemes may in principle be employed [27]. Alternatively, one may adopt the simple and robust approximation mentioned in the same reference. Specifically, we approximate B_k in eq. (3.42) by

$$B_k \approx \text{BFGS}(\{s_{k_i}, \bar{y}_{k_i} + \mu_k s_{k_i}\}_{i=0}^{p-1}) - \mu_k I, \quad (3.44)$$

so that $B_k + \mu_k I \approx \text{BFGS}(\{s_{k_i}, \bar{y}_{k_i} + \mu_k s_{k_i}\}_{i=0}^{p-1})$. Its inverse can then be computed using the standard two-loop recursion. Although this approximation is inexact, it has been reported to yield

nearly identical practical performance while remaining comparatively simple to implement [27]. We adopt this approximation in the this work.

Separately, there exist function-based modified secant conditions [2, 24, 54, 58] that adjust the secant equation using available function values and gradients when those quantities are deemed reliable. Such adjustments aim to improve practical convergence behavior. We partially adopt these function-based modifications following prior work [55, 59]. The suffix -MS in section 3.5 indicates the use of this technique. Because these modifications are well established, we omit low-level implementation details here. Further specifics are available in the cited references and in our open-source implementation^{*1}.

3.4.2 Step Size α_k

We next discuss the adjustment of the step size α_k in Line 4 of Algorithm 2.

As indicated in the pseudo-code, we choose α_k using interpolation of the objective function. Moré–Thuente line searches [35] are widely used in quasi-Newton methods [49], and they typically employ cubic or quadratic interpolation to select trial step sizes. In our implementation, we use essentially the same procedure with a slight modification, namely, clipping the trial step size to the interval $[\beta_{\min}\alpha_k, \beta_{\max}\alpha_k]$. For all numerical experiments, we set the Armijo parameter to $c = 10^{-4}$ and choose the interpolation bounds as $\beta_{\min} = 1/16$ and $\beta_{\max} = 15/16$.

Furthermore, when $\mu_k > 0$ and $\alpha_k = 1$, we introduce a one-time adjustment of the trial step size using gradient information. Let d_k be the search direction and g_k and g_{try} the gradients at the current and trial points. If the directional derivative along d_k changes sign, $d_k^\top g_k < 0$ and $d_k^\top g_{\text{try}} > 0$, and the trial gradient is sufficiently aligned with d_k , namely $d_k^\top g_{\text{try}} > 0.5\|d_k\|\|g_{\text{try}}\|$, we rescale α_k once by a secant-like factor,

$$\alpha_k \leftarrow \text{clip}\left(\alpha_k \frac{-d_k^\top g_k}{d_k^\top g_{\text{try}} - d_k^\top g_k}, \beta_{\min}\alpha_k, \beta_{\max}\alpha_k\right),$$

where $\text{clip}(x, a, b) := \min(\max(x, a), b)$ is a clipping function. When objective values are unreliable and backtracking and interpolation do not occur, this correction effectively refines the step size. Since this adjustment is at most once per iteration, the theoretical results in section 3.3 remain valid.

3.4.3 Regularization Parameter μ_k

We finally discuss the adjustment of the regularization parameter μ_k in Line 8 of Algorithm 1.

In our implementation, we choose the regularization parameter as

$$\mu_k = \text{clip}\left(\frac{1}{10}\|g_k\|, \frac{1}{100}G_k, G_k\right) \text{ where } G_k := \sqrt{\varsigma + \sum_{j \in K, j \leq k} \|g_j\|^2},$$

which is consistent with the general form (eq. (3.7)) with an adaptive factor $\theta_k \in [10^{-2}, 1]$. In practice, we may set $\varsigma = 0$ as long as the initial gradient is nonzero since ς is mainly introduced for avoiding zero division, but to make it consistent with the theory, we set $\varsigma = 10^{-10}$ in all experiments.

When B_k is a good approximation of the Hessian, the regularization parameter μ_k should be small for fast convergence. Since the quantity G_k is strictly increasing with k , it is natural in practice to decrease the effective regularization when sufficient progress in the objective function is achieved. When the difference between the two sides of eq. (3.6) is larger than 1, we restart the accumulation by resetting K^+ to the empty set and G_k to ς . As long as the number of such restarts is finite, the overall convergence analysis in section 3.3 remains valid. Since these choices

are heuristic, there may be room for further improvement. Especially, further increasing μ_k stabilizes the method when the objective function values are highly noisy, and decreasing μ_k more aggressively may accelerate convergence when the objective function is accurate.

3.5 Numerical Experiments

3.5.1 Methods

We compared the following optimization algorithms:

- (Ours): Our proposed regularized quasi-Newton method
 - We used Algorithms 1 and 2 with the details in section 3.4.
 - (Ours-MS) means the variant incorporating the modified secant condition described in section 3.4.1.
- (Line): Line-search L-BFGS method
 - Our Python implementation ported from LBFGSpp [39].
 - The step size was determined using the Moré–Thuente line search, closely resembling SciPy’s implementation.
 - (Line-MS) means the same variant as Ours-MS.
- (Reg): Regularized L-BFGS method [27]
 - A quadratic regularization-based quasi-Newton method that does not rely on line search and is conceptually closer to trust-region methods.
 - We wrapped the function `regLBFGS` using `solveNonmonotone` from [27].
 - (Reg-Sec) means the incorporation of the approximated computation as in eq. (3.44). This is wrapped function of `regLBFGSsec` from [27].
- (SciPy): SciPy’s L-BFGS-B method [49]
 - An established L-BFGS method implemented in C and called from Python.
 - We used the default parameters, with the exception that `ftol` was set to 0 to avoid premature termination.
- (NTQN): Noise-tolerant quasi-Newton method [45, 53]
 - A method designed to accommodate inexact gradient information, as mentioned in section 3.1.1.
 - We mainly used the default parameter settings, and slightly modified the stopping criteria to terminate at the desired gradient norm tolerance.

In the following, each method is referred to by the name given in parentheses. All these methods are L-BFGS-based, and we set the number of stored curvature pairs to 10, the default value in SciPy’s L-BFGS-B implementation.

All experiments were conducted on a Microsoft Windows 11 Home system (version 10.0, build 26100) equipped with a 13th Gen Intel® Core™ i7-1360P CPU, featuring 12 physical cores and 16 logical processors. All computations were performed using the CPU only.

3.5.2 Experimental Results

Test Problems and Settings

We evaluated all algorithms on unconstrained problems from the CUTEst collection [18], implemented in Python using the PyCUTEst interface [16]. All problems were initialized with the default starting points provided by the library. Following the previous work [27], we excluded 17 problems where either the initial point is already stationary or all algorithms are known to fail. This left 220 test problems. To simulate different numerical environments, we tested the following four settings:

explanation	min abs value	relative error	ε_f
64-bit double precision	2.23×10^{-308}	$2^{-52} \approx 2.22 \times 10^{-16}$	2.22×10^{-9}
32-bit single precision	1.18×10^{-38}	$2^{-23} \approx 1.19 \times 10^{-7}$	1.19×10^{-3}
16-bit half precision	6.10×10^{-5}	$2^{-10} \approx 9.77 \times 10^{-4}$	9.77×10^{-2}

Table 3.1: The “min abs value” is the smallest positive normalized number, the “relative error” is the maximum relative rounding error, and ε_f is the parameter in (??) used in our experiments.

- Artificial noise with absolute noise level of 10^{-3} at 64-bit precision (220 problems, $\varepsilon_f = 10^{-2}$)
- 64-bit floating-point precision (220 problems, $\varepsilon_f = 2.22 \times 10^{-9}$)
- 32-bit floating-point precision (220 problems, $\varepsilon_f = 1.19 \times 10^{-2}$)
- 16-bit floating-point precision (152 problems, $\varepsilon_f = 9.77 \times 10^{-1}$)

Each setting uses a different error rate ε_f in the error model (eq. (3.1)), as summarized in Table 3.1. The table relates the machine epsilon (maximum relative rounding error) for each floating-point precision to the ε_f value used in our experiments. We set ε_f equal to or larger than the machine epsilon to account for additional uncertainties in function and gradient evaluations. We also set the maximum number of iterations to $k_{\max} = 15,000$ and timed out runs exceeding 10 minutes per problem.

In the reduced precision settings, to simulate lower numerical precision, we first casted the input vector x to the target lower precision, then computed $\bar{f}(x)$ and $\nabla \bar{f}(x)$ using the lower-precision input. Although the internal computations of CUTEst remain in 64-bit precision, the casting of the input vector effectively simulated the impact of reduced precision on function and gradient evaluations. In the 16-bit precision setting, we excluded problems where the initial gradient norm contains NaN or INF values after casting, resulting in a total of 152 problems.

In the artificial noise setting, we added uniform random noise sampled from $[-10^{-3}, 10^{-3}]$ to both the objective function values and each component of the gradient vectors independently, i.e.,

$$\bar{f}(x) = f(x) + \text{unif}(-10^{-3}, 10^{-3}), \quad \nabla \bar{f}(x) = \nabla f(x) + \text{unif}(-10^{-3}, 10^{-3}) \mathbb{1},$$

where $\text{unif}(a, b)$ denotes a uniform random variable in the interval $[a, b]$ and $\mathbb{1}$ is the vector of all ones. This setting is the same as the one in [45].

Performance Profiles

We used performance profiles [14] to compare the algorithms. Let P denote the set of test problems and S the set of solvers. For each problem $p \in P$ and solver $s \in S$, let $t_{p,s} > 0$ denote the number of function and gradient evaluations required to reach a specified gradient norm tolerance $\varepsilon_{\text{gtol}}$. Note that $t_{p,s}$ is not time in this experiment but the number of evaluations. When a solver fails to converge within the gradient tolerance or exceeds a maximum number of evaluations, we set $t_{p,s} = +\infty$. Then, the performance ratio $r_{p,s}$ is defined as

$$r_{p,s} = \frac{t_{p,s}}{\min_{s' \in S} t_{p,s'}}.$$

For a given threshold $\tau \geq 1$, the performance profile plots the proportion of problems for which $r_{p,s} \leq \tau$, which means that the y -coordinate for solver s at x -coordinate τ is given by

$$\rho_s(\tau) = \frac{1}{|P|} |\{p \in P \mid r_{p,s} \leq \tau\}|.$$

A curve that lies above others indicates better overall performance.

Results with Artificial Noise

We first present the results in the artificial noise setting described in section 3.5.2. Figure 3.1 presents the performance profile. In this regime, the proposed method shows superior robustness compared to other quasi-Newton methods. This result highlights the validity of our theoretical analysis in section 3.3 and demonstrates the effectiveness of our regularization-based approach in handling noisy objective function values.

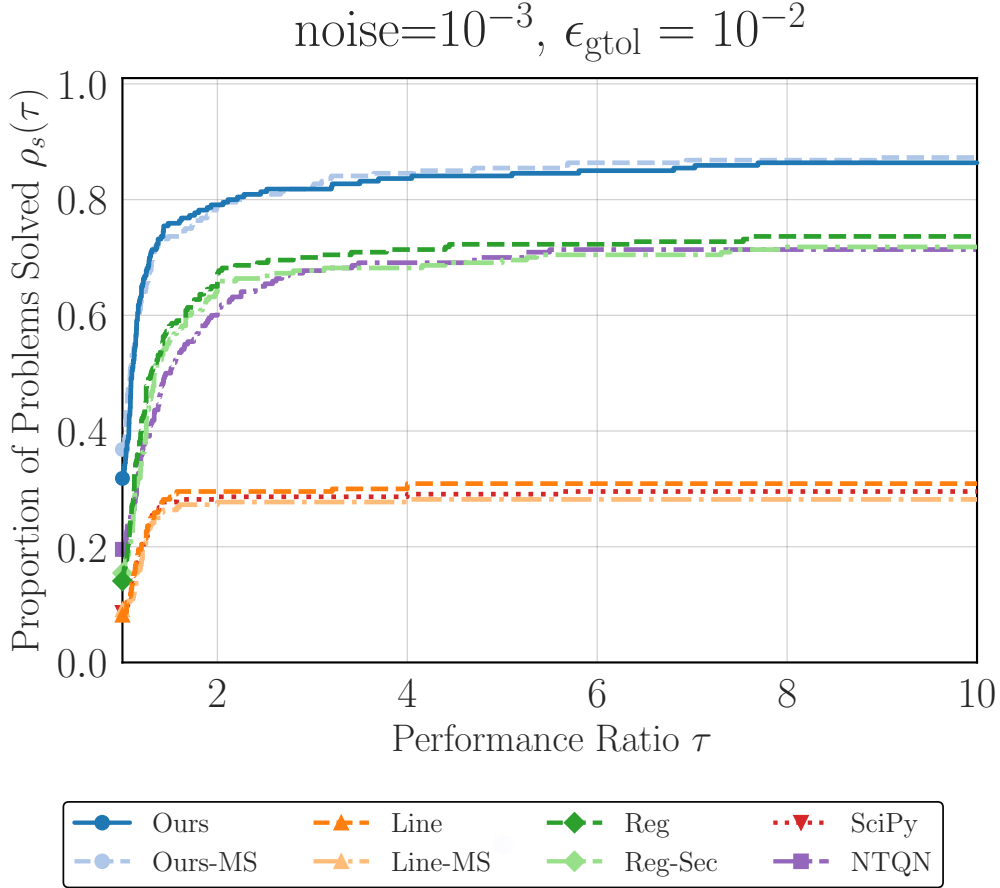


Fig. 3.1: Performance profile for artificial noise setting (level of 10^{-3}) and $\epsilon_{\text{gtol}} = 10^{-2}$.

Results at Different Precision Levels

We next present performance profiles for each floating-point precision (64-bit, 32-bit, and 16-bit). We evaluated each method at three gradient norm tolerance levels, i.e., $\epsilon_{\text{gtol}} \in \{10^{-1}, 10^{-3}, 10^{-5}\}$. These tolerance levels represent different optimization regimes, from moderate accuracy to high precision requirements. Figure 3.2 shows the comprehensive performance profiles across all three precision levels. Our methods (Ours) and (Ours-MS) demonstrates competitive or superior performance compared to existing methods in the standard 64-bit setting. As numerical precision degrades from 64-bit to 32-bit and further to 16-bit, our regularization-based approach demonstrates improved relative performance. The proposed method maintains stability even when traditional line-search methods encounter difficulties with reduced precision arithmetic.

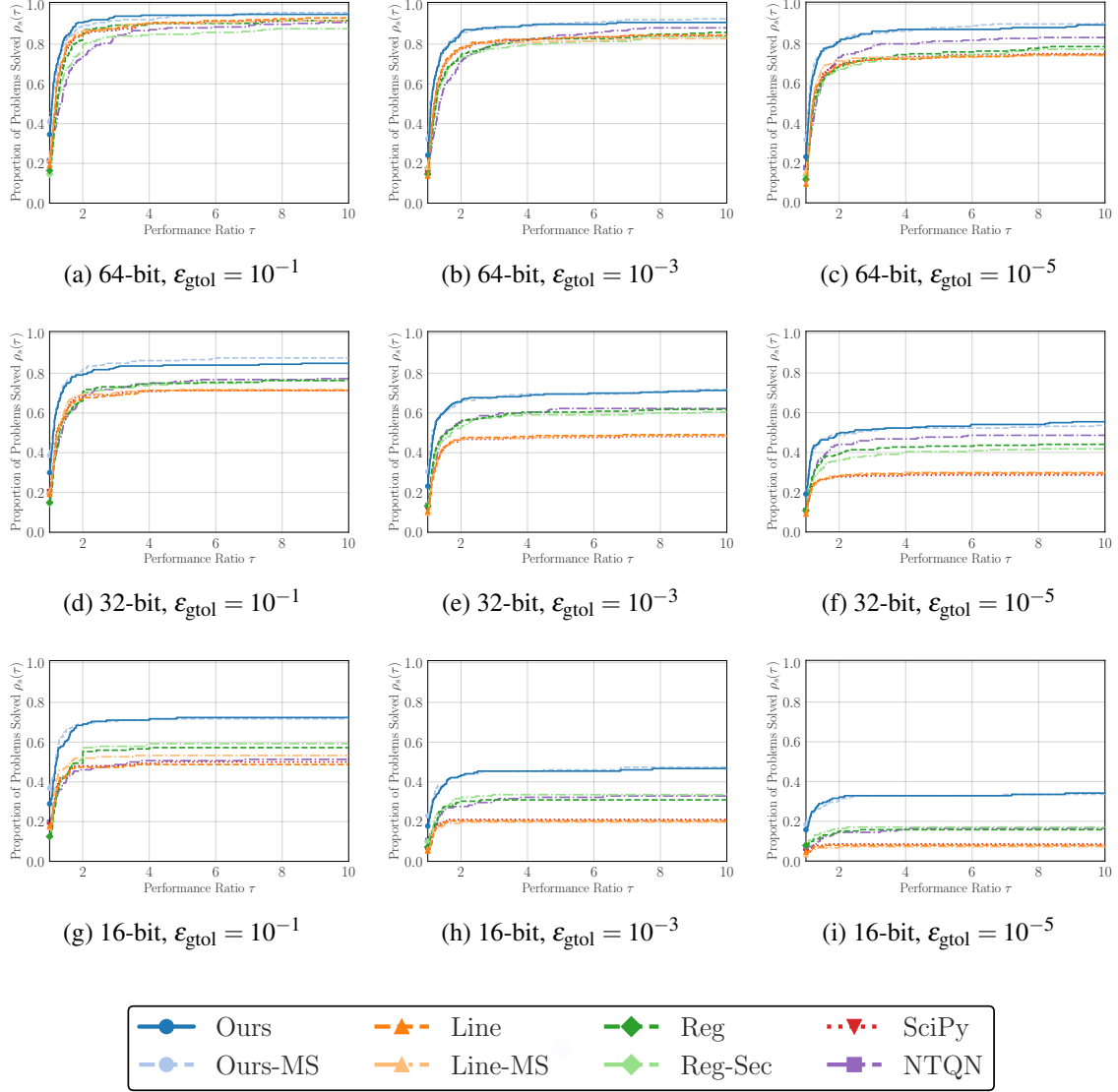


Fig. 3.2: Performance profiles across different floating-point precisions (64, 32, and 16-bit) and tolerance levels. Each row corresponds to a different precision level, demonstrating the robustness of the proposed method as numerical precision degrades.

3.5.3 Computational Time per Iteration

In this subsection, we present an experiment to compare the per-iteration computational cost of each method. Since the previous experiments focused on the number of function and gradient evaluations, they did not directly reflect the computational overhead of each algorithm. Here, we measured the total execution time required to perform a fixed number of iterations on a simple problem. We conducted experiments on the ill-conditioned quadratic problem

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \quad \frac{1}{2} \sum_{i=1}^n i x_i^2$$

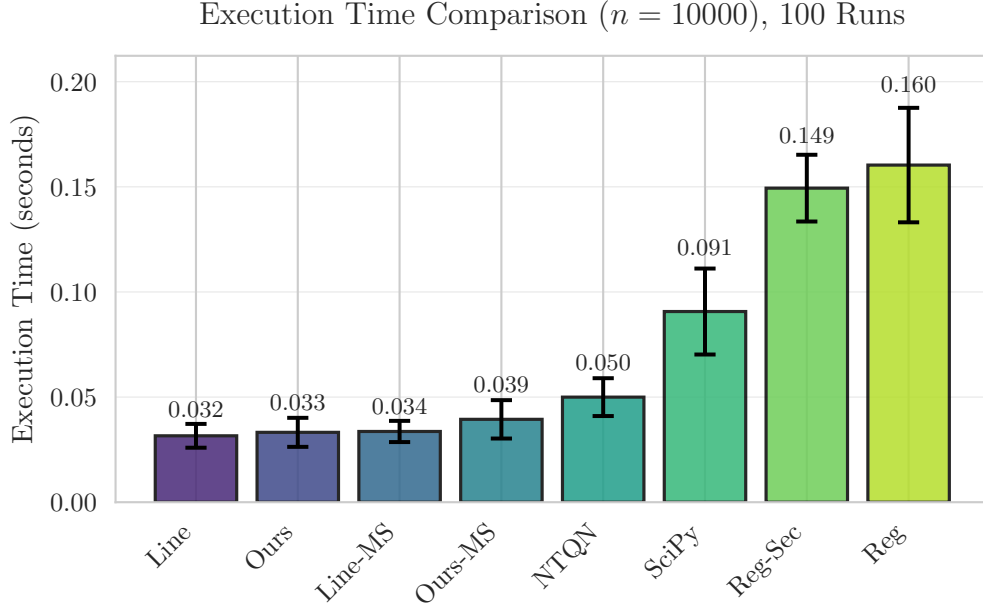


Fig. 3.3: Mean execution times for the experiment in section 3.5.3. This plot roughly indicates the extra computational overhead of each method. Error bars represent one standard deviation over 100 runs.

with $n = 10,000$, and measured the total execution time. The initial point x_0 was chosen as the all-ones vector. We used memory size $m = 10$ and maximum iterations $k_{\max} = 100$, and we did not set any stopping criteria other than reaching $k_{\max}$. The simplicity of the optimization problem and the small value of $k_{\max}$ are well suited to the purpose of this experiment, as they ensure that the total runtime is dominated by algorithmic overhead rather than function or gradient computation. Timing data were collected after three warm-up runs, followed by 100 recorded trials.

In Figure 3.3, we report the execution time statistics, and we can observe that our proposed methods (Ours) and (Ours-MS) exhibit competitive performance compared to other methods. Combining with the results from section 3.5.2, our methods is competitive or superior in total oracle calls and per-iteration computational cost, indicating practical efficiency. As a side note, although the (SciPy) calls optimized C routines, it incurs associated dispatch overhead, producing a larger mean time. The main reason of the (Reg) and (Reg-Sec) methods taking longer execution time is mainly due to the implementation aspect of the original code.

In Table 3.2, we report the total number of oracle calls (calls of $\bar{f}$ and $\nabla \bar{f}$) and final objective values $\bar{f}(x_{k_{\max}})$. The purpose of this table is to confirm that all methods achieved similar convergence behavior within the limited number of iterations. The comparable values in the table indicate that the comparison of execution times is fair and not affected by differences in convergence behavior.

	Line	Ours	Line-MS	Ours-MS	NTQN	SciPy	Reg-Sec	Reg
# Calls	212	202	212	202	219	212	206	206
$\bar{f}(x_{k_{\max}})$	1.24	1.34	1.24	1.34	1.43	1.24	1.48	1.47

Table 3.2: The number of oracle calls and final observed objective values for the experiment in section 3.5.3.

3.6 Conclusion

In this work, we proposed a regularized quasi-Newton method for noisy nonconvex optimization and established its global convergence properties. We also demonstrated the method's effectiveness through numerical experiments on standard benchmark problems under various noise and precision settings. The results indicate that our method is robust and practically efficient, particularly in noisy or low-precision environments where traditional line-search methods may struggle.

Future work includes the investigation of local convergence properties, the extension to constrained optimization, and the application to practical problems in machine learning and scientific computing. Additionally, exploring adaptive strategies for selecting algorithmic parameters based on problem characteristics could further enhance the method's performance.

Explicitly analyzing the impact of inexact gradient evaluations can also provide valuable insights into the method's robustness and guide further enhancements.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Professor Akiko Takeda, and Project Research Associate Naoki Marumo for their extensive guidance on the direction and content of the research.

Bibliography

- [1] Babaie-Kafaki S (2011) A modified BFGS algorithm based on a hybrid secant equation. *Science China Mathematics* 54(9):2019–2036, DOI 10.1007/s11425-011-4232-7
- [2] Babaie-Kafaki S, Ghanbari R, Mahdavi-Amiri N (2010) Two new conjugate gradient methods based on modified secant equations. *Journal of Computational and Applied Mathematics* 234(5):1374–1386, DOI 10.1016/j.cam.2010.01.052
- [3] Baillon JB, Haddad G (1977) Quelques propriétés des opérateurs angle-bornés et ϵ -cycliquement monotones. *Israel Journal of Mathematics* 26(2):137–150, DOI 10.1007/BF03007664
- [4] Bauschke HH, Combettes PL (2009) The Baillon-Haddad Theorem Revisited. DOI 10.48550/arXiv.0906.0807, [0906.0807](#)
- [5] Bellavia S, Gurioli G, Morini B, Toint PL (2021) The Impact of Noise on Evaluation Complexity: The Deterministic Trust-Region Case. DOI 10.48550/arXiv.2104.02519, [2104.02519](#)
- [6] Berahas AS, Cao L, Choromanski K, Scheinberg K (2021) A Theoretical and Empirical Comparison of Gradient Approximations in Derivative-Free Optimization. DOI 10.48550/arXiv.1905.01332, [1905.01332](#)
- [7] Berahas AS, Curtis FE, Zhou B (2022) Limited-memory BFGS with displacement aggregation. *Mathematical Programming* 194(1):121–157, DOI 10.1007/s10107-021-01621-6
- [8] Byrd RH, Nocedal J (1989) A Tool for the Analysis of Quasi-Newton Methods with Application to Unconstrained Minimization. *SIAM Journal on Numerical Analysis* 26(3):727–739, DOI 10.1137/0726042
- [9] Caffisch RE (1998) Monte Carlo and quasi-Monte Carlo methods. *Acta Numerica* 7:1–49, DOI 10.1017/S0962492900002804
- [10] Carter RG (1993) Numerical Experience with a Class of Algorithms for Nonlinear Optimization Using Inexact Function and Gradient Information. *SIAM Journal on Scientific Computing* 14(2):368–388, DOI 10.1137/0914023
- [11] Clancy RJ, Menickelly M, Hückelheim J, Hovland P, Nalluri P, Gjini R (2022) TROPHY: Trust Region Optimization Using a Precision Hierarchy. In: *Computational Science – ICCS 2022*, Springer, Cham, pp 445–459, DOI 10.1007/978-3-031-08751-6_32
- [12] Corless RM, Gonnet GH, Hare DEG, Jeffrey DJ, Knuth DE (1996) On the LambertW function. *Advances in Computational Mathematics* 5(1):329–359, DOI 10.1007/BF02124750
- [13] Doikov N, Mishchenko K, Nesterov Y (2024) Super-Universal Regularized Newton Method. *SIAM Journal on Optimization* 34(1):27–56, DOI 10.1137/22M1519444
- [14] Dolan ED, Moré JJ (2002) Benchmarking optimization software with performance profiles. *Mathematical Programming* 91(2):201–213, DOI 10.1007/s101070100263
- [15] Duchi J, Hazan E, Singer Y (2011) Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. *Journal of Machine Learning Research* 12(61):2121–2159
- [16] Fowkes J, Roberts L, Búrmen Á (2022) PyCUTEst: An open source Python package of optimization test problems. *Journal of Open Source Software* 7(78):4377, DOI 10.21105/joss.04377
- [17] Gao W, Goldfarb D (2018) Quasi-Newton Methods: Superlinear Convergence Without Line Searches for Self-Concordant Functions. DOI 10.48550/arXiv.1612.06965, [1612.06965](#)

- [18] Gould NI, Orban D, Toint PL (2015) CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for mathematical optimization. *Comput Optim Appl* 60(3):545–557, DOI 10.1007/s10589-014-9687-3
- [19] Gower RM, Goldfarb D, Richtárik P (2016) Stochastic Block BFGS: Squeezing More Curvature out of Data. DOI 10.48550/arXiv.1603.09649, [1603.09649](#)
- [20] Grapiglia GN, Stella GFD (2022) An adaptive trust-region method without function evaluations. *Computational Optimization and Applications* 82(1):31–60, DOI 10.1007/s10589-022-00356-0
- [21] Gratton S, Toint PL (2020) A note on solving nonlinear optimization problems in variable precision. *Computational Optimization and Applications* 76(3):917–933, DOI 10.1007/s10589-020-00190-2
- [22] Gratton S, Toint PL (2023) OFFO minimization algorithms for second-order optimality and their complexity. *Computational Optimization and Applications* 84(2):573–607, DOI 10.1007/s10589-022-00435-2
- [23] Gratton S, Jerad S, Toint PhL (2024) Complexity of a class of first-order objective-function-free optimization algorithms. *Optimization Methods and Software* 0(0):1–31, DOI 10.1080/10556788.2023.2296431
- [24] Hassan BA, Moghrabi IAR (2023) A modified secant equation quasi-Newton method for unconstrained optimization. *Journal of Applied Mathematics and Computing* 69(1):451–464, DOI 10.1007/s12190-022-01750-x
- [25] Higham NJ (2002) *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, DOI 10.1137/1.9780898718027
- [26] Izycheva A, Darulova E (2017) On sound relative error bounds for floating-point arithmetic. In: *Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design, FMCAD Inc, Austin, Texas*
- [27] Kanzow C, Steck D (2023) Regularization of limited memory quasi-Newton methods for large-scale nonconvex minimization. *Mathematical Programming Computation* 15(3):417–444, DOI 10.1007/s12532-023-00238-4
- [28] Lee M, Sun Y (2025) Advancing Multi-Secant Quasi-Newton Methods for General Convex Functions. DOI 10.48550/arXiv.2504.07330, [2504.07330](#)
- [29] Li DH, Fukushima M (2001) On the Global Convergence of the BFGS Method for Nonconvex Unconstrained Optimization Problems. *SIAM Journal on Optimization* 11(4):1054–1064, DOI 10.1137/S1052623499354242
- [30] Li YJ, Li DH (2009) Truncated regularized Newton method for convex minimizations. *Computational Optimization and Applications* 43(1):119–131, DOI 10.1007/s10589-007-9128-7
- [31] Liu DC, Nocedal J (1989) On the limited memory BFGS method for large scale optimization. *Mathematical Programming* 45(1):503–528, DOI 10.1007/BF01589116
- [32] Lotfi S, de Ruisselet TB, Orban D, Lodi A (2020) Stochastic Damped L-BFGS with Controlled Norm of the Hessian Approximation. DOI 10.13140/RG.2.2.27851.41765/1, [2012.05783](#)
- [33] Mannel F (2025) A Globalization of L-BFGS and the Barzilai–Borwein Method for Nonconvex Unconstrained Optimization. *Journal of Optimization Theory and Applications* 204(3):1–34, DOI 10.1007/s10957-024-02565-5
- [34] Mannel F, Om Aggrawal H, Modersitzki J (2024) A structured L-BFGS method and its application to inverse problems. *Inverse Problems* 40(4):045022, DOI 10.1088/1361-6420/ad2c31
- [35] Moré JJ, Thuente DJ (1994) Line search algorithms with guaranteed sufficient decrease. *ACM Trans Math Softw* 20(3):286–307, DOI 10.1145/192115.192132
- [36] Moré JJ, Wild SM (2011) Estimating Computational Noise. *SIAM Journal on Scientific Computing* 33(3):1292–1314, DOI 10.1137/100786125
- [37] Nesterov Y, Polyak B (2006) Cubic regularization of Newton method and its global perfor-

- mance. *Mathematical Programming* 108(1):177–205, DOI 10.1007/s10107-006-0706-8
- [38] Nocedal J, Wright SJ (1999) *Numerical Optimization*. Springer, DOI 10.1007/978-0-387-40065-5
- [39] Okazaki N (2007) libLBFGS: A library of limited-memory broyden-fletcher-goldfarb-shanno (l-BFGS)
- [40] Paquette C, Scheinberg K (2020) A Stochastic Line Search Method with Expected Complexity Analysis. *SIAM Journal on Optimization* 30(1):349–376, DOI 10.1137/18M1216250
- [41] Powell MJ (1978) Algorithms for nonlinear constraints that use lagrangian functions. *Math Program* 14(1):224–248, DOI 10.1007/BF01588967
- [42] Ranganath A, Singhal M, Marcia R (2025) Symmetric Rank-One Quasi-Newton Methods for Deep Learning Using Cubic Regularization. DOI 10.48550/arXiv.2502.12298, [2502.12298](#)
- [43] Ruan H, Yang W (2024) Adaptive Regularized Quasi-Newton Method Using Inexact First-Order Information. *Journal of Computational Mathematics* 42(6):1656–1687, DOI 10.4208/jcm.2306-m2022-0279
- [44] Sheng Z, Yuan G, Cui Z (2018) A new adaptive trust region algorithm for optimization problems. *Acta Mathematica Scientia* 38(2):479–496, DOI 10.1016/S0252-9602(18)30762-8
- [45] Shi HJM, Xie Y, Byrd R, Nocedal J (2022) A Noise-Tolerant Quasi-Newton Algorithm for Unconstrained Optimization. *SIAM Journal on Optimization* 32(1):29–55, DOI 10.1137/20M1373190
- [46] Sun S, Nocedal J (2023) A trust region method for noisy unconstrained optimization. *Mathematical Programming* 202(1):445–472, DOI 10.1007/s10107-023-01941-9
- [47] Ueda K, Yamashita N (2010) Convergence Properties of the Regularized Newton Method for the Unconstrained Nonconvex Optimization. *Applied Mathematics and Optimization* 62(1):27–46, DOI 10.1007/s00245-009-9094-9
- [48] Ueda K, Yamashita N (2014) A regularized Newton method without line search for unconstrained optimization. *Computational Optimization and Applications* 59(1):321–351, DOI 10.1007/s10589-014-9656-x
- [49] Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, Burovski E, Peterson P, Weckesser W, Bright J, van der Walt SJ, Brett M, Wilson J, Millman KJ, Mayorov N, Nelson ARJ, Jones E, Kern R, Larson E, Carey CJ, Polat I, Feng Y, Moore EW, VanderPlas J, Laxalde D, Perktold J, Cimrman R, Henriksen I, Quintero EA, Harris CR, Archibald AM, Ribeiro AH, Pedregosa F, van Mulbregt P, SciPy 10 Contributors (2020) SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods* 17:261–272, DOI 10.1038/s41592-019-0686-2
- [50] Wang S, Fadili J, Ochs P (2025) Convergence rates of regularized quasi-Newton methods without strong convexity. DOI 10.48550/arXiv.2506.00521, [2506.00521](#)
- [51] Ward R, Wu X, Bottou L (2020) AdaGrad stepsizes: Sharp convergence over nonconvex landscapes. *J Mach Learn Res* 21(1):219:9047–219:9076
- [52] Wei Z, Li G, Qi L (2006) New quasi-Newton methods for unconstrained optimization problems. *Applied Mathematics and Computation* 175(2):1156–1188, DOI 10.1016/j.amc.2005.08.027
- [53] Xie Y, Byrd RH, Nocedal J (2020) Analysis of the BFGS method with errors. *SIAM Journal on Optimization* 30(1):182–209
- [54] Yabe H, Ogasawara H, Yoshino M (2007) Local and superlinear convergence of quasi-Newton methods based on modified secant conditions. *Journal of Computational and Applied Mathematics* 205(1):617–632, DOI 10.1016/j.cam.2006.05.018
- [55] Yuan G, Wei Z (2010) Convergence analysis of a modified BFGS method on convex minimizations. *Computational Optimization and Applications* 47(2):237–255, DOI 10.1007/s10589-008-9219-0
- [56] Yuan YX (1991) A Modified BFGS Algorithm for Unconstrained Optimization. *IMA Journal of Numerical Analysis* 11(3):325–332, DOI 10.1093/imanum/11.3.325

- [57] Zhang H, Ni Q (2018) A new regularized quasi-Newton method for unconstrained optimization. *Optimization Letters* 12(7):1639–1658, DOI 10.1007/s11590-018-1236-z
- [58] Zhang J, Xu C (2001) Properties and numerical performance of quasi-Newton methods with modified quasi-Newton equations. *Journal of Computational and Applied Mathematics* 137(2):269–278, DOI 10.1016/S0377-0427(00)00713-5
- [59] Zhang JZ, Deng NY, Chen LH (1999) New Quasi-Newton Equation and Related Methods for Unconstrained Optimization. *Journal of Optimization Theory and Applications* 102(1):147–167, DOI 10.1023/A:1021898630001

A

Appendix

A.1 Proof of Lemma 5

The proof of Lemma 5 is similar to existing results [51, Lemma 3.2], [23, Lemma 3.1].

Proof. Recall that eq. (3.7) defines μ_k as $\theta_k \sqrt{\varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2}$, and Line 8 of Algorithm 1 asserts $\theta_k \in [\theta_{\min}, \theta_{\max}]$. Let $k_i \in K^+$ be the i -th element of K^+ . For any $0 < x < y$, $x/y \leq -\ln(1 - x/y) = \ln y - \ln(y - x)$ holds. Thus, we have

$$\begin{aligned} \frac{\|g_{k_i}\|^2}{\mu_{k_i}^2} &\leq \frac{1}{\theta_{\min}^2} \frac{\|g_{k_i}\|^2}{\varsigma + \sum_{\ell=0}^i \|g_{k_\ell}\|^2} && \text{(definition)} \\ &\leq \frac{1}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{\ell=0}^i \|g_{k_\ell}\|^2 \right) - \ln \left(\varsigma + \sum_{\ell=0}^{i-1} \|g_{k_\ell}\|^2 \right) \right). && (x/y \leq \ln y - \ln(y - x)) \end{aligned}$$

Similarly, $x/\sqrt{y} \geq x/(\sqrt{y} + \sqrt{y - x}) = \sqrt{y} - \sqrt{y - x}$ holds. Thus, we have

$$\begin{aligned} \frac{\|g_{k_i}\|^2}{\mu_{k_i}} &\geq \frac{1}{\theta_{\max}} \frac{\|g_{k_i}\|^2}{\sqrt{\varsigma + \sum_{\ell=0}^i \|g_{k_\ell}\|^2}} && \text{(definition)} \\ &\geq \frac{1}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{\ell=0}^i \|g_{k_\ell}\|^2} - \sqrt{\varsigma + \sum_{\ell=0}^{i-1} \|g_{k_\ell}\|^2} \right). && (x/\sqrt{y} \geq \sqrt{y} - \sqrt{y - x}) \end{aligned}$$

Summing up these inequalities over $0 \leq i < |K^+|$ yields the claim. $\square$

A.2 Proof of Proposition 2

The proof of Proposition 2 is similar to existing results [17, Theorem 5.5] [19, Lemma 1].

Proof. We consider standard BFGS updates with a single curvature pair (s, y) . From eqs. (3.39) and (3.40), we have

$$\left\| \frac{yy^\top}{y^\top s} \right\| = \frac{y^\top y}{y^\top s} \leq \Lambda, \quad \left\| \frac{ys^\top}{y^\top s} \right\| \leq \frac{\|y\| \|s\|}{\sqrt{\frac{1}{\Lambda} \|y\|^2} \sqrt{\lambda \|s\|^2}} = \sqrt{\kappa}, \quad \left\| \frac{ss^\top}{y^\top s} \right\| = \frac{s^\top s}{y^\top s} \leq \frac{1}{\lambda}. \quad (\text{A.1})$$

Let B, H denote the positive definite Hessian approximation before the update and its inverse, and B_+, H_+ the updated matrices. The BFGS update rule is

$$B_+ = B - \frac{Bss^\top B}{s^\top Bs} + \frac{yy^\top}{y^\top s}, \quad H_+ = \left(I - \frac{sy^\top}{y^\top s}\right) H \left(I - \frac{ys^\top}{y^\top s}\right) + \frac{ss^\top}{y^\top s}.$$

Since B is symmetric positive definite matrix and $s \neq 0$, we have

$$\|B_+\| \leq \|B\| + \left\| \frac{yy^\top}{y^\top s} \right\| \leq \|B\| + \Lambda, \quad (\text{A.2})$$

$$\begin{aligned} \|H_+\| &= \left\| \left(I - \frac{ys^\top}{y^\top s}\right)^\top H \left(I - \frac{ys^\top}{y^\top s}\right) + \frac{ss^\top}{y^\top s} \right\| \\ &\leq \left\| I - \frac{ys^\top}{y^\top s} \right\|^2 \|H\| + \left\| \frac{ss^\top}{y^\top s} \right\| \\ &\leq \left(1 + \left\| \frac{ys^\top}{y^\top s} \right\|\right)^2 \|H\| + \left\| \frac{ss^\top}{y^\top s} \right\| \\ &\leq (1 + \sqrt{\kappa})^2 \|H\| + \frac{1}{\lambda}. \end{aligned} \quad (\text{eq. (A.1)}) \quad (\text{A.3})$$

Applying eqs. (A.2) and (A.3) recursively over p updates and using the initial matrix γI , we obtain

$$\begin{aligned} \left\| \text{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1}) \right\| &\leq \gamma + p\Lambda, \\ \left\| (\text{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1}))^{-1} \right\| &\leq (1 + \sqrt{\kappa})^{2p} \gamma + \frac{1}{\lambda} \sum_{i=0}^{p-1} (1 + \sqrt{\kappa})^{2i} \\ &= (1 + \sqrt{\kappa})^{2p} \gamma + \frac{1}{\lambda} \frac{(1 + \sqrt{\kappa})^{2p} - 1}{(1 + \sqrt{\kappa})^2 - 1} \\ &\leq (1 + \sqrt{\kappa})^{2p} \left(\gamma + \frac{1}{\lambda(2\sqrt{\kappa} + \kappa)} \right). \end{aligned}$$

Therefore, $\text{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1})$ satisfies the stated bounds in eq. (3.41). $\square$

The conditions eqs. (3.39) and (3.40) are standard in the analysis of quasi-Newton methods [8, Theorem 2.1]. If f is L -smooth (Assumption 2) and convex, then by the Baillon–Haddad theorem [3, 4] its gradient is $1/L$ -cocoercive:

$$(\nabla f(x) - \nabla f(y))^\top (x - y) \geq \frac{1}{L} \|\nabla f(x) - \nabla f(y)\|^2 \quad (\text{A.4})$$

for all $x, y \in \mathbb{R}^n$. Applying this inequality with $x = x_{k+1}$ and $y = x_k$ yields eq. (3.39) with $\Lambda = L$.